

# INFORME FINAL DE PROYECTO DE TÍTULO

Proyecto APT

## POINTCHECK

Sistema Integral de Gestión de Reservas, Atención de Clientes y  
Administración para Prestadores de Servicios

### Integrantes

Ismael Jiménez

Benjamín Devia

Máximo Zúñiga

### Carrera

Analista Programador

### Asignatura

Portafolio de Título

### Profesor Guía

Fabio Andrés Vivas Cabrera

### Institución

Duoc UC – Plaza Oeste

### Año

2026

# ÍNDICE GENERAL

## INTRODUCCIÓN

- Problemática y digitalización de procesos administrativos
- Presentación del Proyecto PointCheck
- Evolución y refactorización a partir del prototipo inicial
- Tecnologías seleccionadas y metodología de trabajo
- Estructura general del informe

## CAPÍTULO I. JUSTIFICACIÓN DEL PROYECTO

- 1.1 Contexto actual de la digitalización empresarial
- 1.2 Problemática operacional en negocios basados en reservas
- 1.3 Justificación técnica y refactorización del prototipo inicial
- 1.4 Relación con el perfil profesional de la Ingeniería en Informática
- 1.5 Enfoque tecnológico y seguridad de la información

## CAPÍTULO II. OBJETIVOS

- 2.1 Objetivo General
- 2.2 Objetivos Específicos
  - 2.2.1 Analizar el prototipo inicial e identificar oportunidades de mejora
  - 2.2.2 Diseñar una arquitectura escalable basada en módulos funcionales
  - 2.2.3 Implementar un backend robusto utilizando Spring Boot
  - 2.2.4 Desarrollar una aplicación móvil moderna utilizando Kotlin y Jetpack Compose
  - 2.2.5 Diseñar una base de datos relacional eficiente
  - 2.2.6 Incorporar mecanismos de seguridad para la protección de la información
  - 2.2.7 Integrar servicios externos para enriquecer la experiencia del usuario
  - 2.2.8 Optimizar el rendimiento y la experiencia de uso de la aplicación
  - 2.2.9 Verificar la calidad del sistema mediante pruebas funcionales
  - 2.2.10 Documentar íntegramente el proceso de desarrollo

## CAPÍTULO III. ALCANCE DEL PROYECTO

- 3.1 Alcance General del Proyecto APT
- 3.2 Funcionalidades Comprometidas por Perfil
  - 3.2.1 Herramientas y funciones para el perfil Cliente
  - 3.2.2 Herramientas y funciones para el perfil Especialista
  - 3.2.3 Herramientas y funciones para el perfil Administrador
  - 3.2.4 Mecanismos complementarios del sistema
- 3.3 Funcionalidades Fuera del Alcance y consideraciones futuras
- 3.4 Beneficiarios directos e indirectos del proyecto

## CAPÍTULO IV. METODOLOGÍA DE DESARROLLO

- 4.1 Selección de la metodología ágil Scrum
- 4.2 Organización y distribución de responsabilidades en el equipo
- 4.2 Gestión del desarrollo mediante Sprints
- 4.4 Herramientas tecnológicas utilizadas en la gestión

## **CAPÍTULO V. PLANIFICACIÓN Y DESARROLLO DEL PROYECTO**

- 5.1 Planificación General e incremental del proyecto
- 5.2 Evolución del proyecto mediante Sprints
  - 5.2.1 Sprint 1: Análisis del prototipo y definición de la arquitectura
  - 5.2.2 Sprint 2: Refactorización estructural y reorganización del proyecto
  - 5.2.3 Sprint 3: Implementación de funcionalidades principales
  - 5.2.4 Sprint 4: Consolidación de reservas y experiencia de usuario
- 5.3 Evolución continua en Sprints posteriores
- 5.4 Resultados finales del proceso de planificación

## **CAPÍTULO VI. DESARROLLO TÉCNICO DEL PROYECTO**

- 6.1 Arquitectura General del Sistema (Modelo Cliente-Servidor)
- 6.2 Evolución e implementación de la *Feature-Based Architecture*
- 6.3 Desarrollo del Backend con Spring Boot y estructura modular
- 6.4 Implementación de la Lógica de Negocio y separación de responsabilidades
- 6.5 Desarrollo del Frontend con Kotlin y Jetpack Compose (Patrón MVVM)
- 6.6 Organización del Frontend por módulos funcionales
- 6.7 Mecanismos de comunicación e integración entre Frontend y Backend
- 6.8 Diseño y evolución del modelo de Base de Datos MySQL
- 6.9 Integración de APIs Externas (OpenWeather API y Google Maps API)
- 6.10 Seguridad del Sistema: Cifrado, control de accesos, UUID y auditoría
- 6.11 Alineación y cumplimiento técnico con la norma ISO/IEC 27001

## **CAPÍTULO VII. VERIFICACIÓN DE LA CALIDAD DEL SISTEMA**

- 7.1 Enfoque y aseguramiento continuo de la calidad del software
- 7.2 Estrategia de verificación por capas de arquitectura
- 7.3 Casos de Prueba Funcionales (Módulos validados)
- 7.4 Errores detectados y mitigados durante el desarrollo
- 7.5 Validación del proceso de integración completa
- 7.6 Resultados consolidados del proceso de calidad

## **CAPÍTULO VIII. RESULTADOS OBTENIDOS**

- 8.1 Resultados Generales y evolución del proyecto
- 8.2 Resultados Técnicos alcanzados
- 8.3 Resultados Funcionales por perfiles de usuario
- 8.4 Resultados e impactos en Seguridad y Calidad
- 8.5 Resultados del proceso de verificación mediante pruebas
- 8.6 Impacto y proyección profesional del proyecto

## **CAPÍTULO IX. CONCLUSIONES Y TRABAJO FUTURO**

- 9.1 Conclusiones del desarrollo técnico y metodológico
- 9.2 Trabajo Futuro: Próximas líneas de desarrollo e innovaciones del sistema

## **CAPÍTULO X. COMPETENCIAS DE EMPLEABILIDAD**

- 10.1 Gestión del Trabajo en Equipo mediante Scrum
- 10.2 Procesos de Análisis y Resolución de Problemas técnicos
- 10.3 Aprendizajes Profesionales adquiridos

## REFERENCIAS BIBLIOGRÁFICAS

## ANEXOS

- Anexo A. Modelo de Datos (Diagrama entidad-relación)
- Anexo B. Diagramas de Arquitectura del Sistema
- Anexo C. Registro de Casos de Prueba Funcionales
- Anexo D. Cronograma y Planificación Scrum
- Anexo E. Evidencias del Desarrollo (Capturas de pantalla de la interfaz)
- Anexo F. Evidencias del Repositorio de Código (GitHub)

# Introducción

El desarrollo de soluciones tecnológicas orientadas a optimizar procesos empresariales constituye actualmente uno de los principales desafíos para la Ingeniería en Informática. La creciente digitalización de las organizaciones ha impulsado la incorporación de herramientas capaces de automatizar procesos administrativos, mejorar la gestión de la información y facilitar la toma de decisiones mediante el acceso oportuno a datos confiables. En este contexto, los sistemas de gestión empresarial han adquirido un rol fundamental dentro de pequeñas y medianas empresas, especialmente aquellas cuyo modelo de negocio depende de la administración eficiente de reservas, atención de clientes y control financiero.

A pesar del avance tecnológico experimentado durante la última década, numerosos negocios continúan administrando sus operaciones mediante procedimientos manuales o utilizando aplicaciones que únicamente resuelven necesidades específicas sin integrar completamente el funcionamiento del negocio. Esta situación genera problemas asociados a la duplicidad de reservas, pérdida de información, escasa trazabilidad de las actividades realizadas, dificultades para administrar múltiples profesionales y una limitada capacidad para analizar el desempeño operativo de la organización.

Considerando esta problemática, el presente Proyecto APT desarrolla **PointCheck**, una plataforma integral diseñada para administrar el ciclo completo de atención de negocios basados en reservas de servicios. La solución incorpora herramientas para clientes, especialistas y administradores, permitiendo gestionar reservas, controlar agendas de trabajo, registrar atenciones, administrar cobros, generar reportes financieros y supervisar la operación del sistema mediante paneles de información diferenciados según el rol del usuario.

El proyecto surge a partir de un prototipo inicial entregado al equipo de desarrollo, cuya funcionalidad permitía demostrar parcialmente la idea del sistema, pero que presentaba importantes limitaciones desde el punto de vista arquitectónico, estructural y de escalabilidad. En consecuencia, durante el desarrollo del Proyecto APT se decidió realizar un proceso de análisis y refactorización completa del software, reorganizando tanto el backend como la aplicación móvil bajo una arquitectura modular basada en funcionalidades (Feature-Based Architecture), incorporando además nuevas tecnologías, mecanismos de seguridad y mejoras orientadas a la mantenibilidad del sistema.

Para la implementación de la solución se utilizaron tecnologías ampliamente utilizadas dentro de la industria del desarrollo de software. La aplicación móvil fue desarrollada utilizando Kotlin y Jetpack Compose bajo el patrón de arquitectura MVVM, mientras que el backend fue implementado mediante Spring Boot utilizando una arquitectura REST para la comunicación entre componentes. La persistencia de la información se realizó

mediante MySQL y Spring Data JPA, permitiendo administrar de forma eficiente los datos generados por la plataforma.

Durante el proceso de desarrollo también se incorporaron mejoras orientadas a fortalecer la calidad del software y la seguridad de la información. Entre ellas destacan la implementación de autenticación segura mediante BCrypt, control de acceso basado en roles, utilización de identificadores UUID, mecanismos de auditoría, operaciones transaccionales mediante @Transactional, optimización de consultas SQL para generación de reportes y mecanismos de almacenamiento local mediante Room para mejorar la experiencia del usuario.

El desarrollo del proyecto fue gestionado utilizando la metodología ágil Scrum, organizando el trabajo mediante Sprints de dos semanas, planificación incremental y control de versiones utilizando GitHub. Esta metodología permitió coordinar el trabajo colaborativo entre los integrantes del equipo, mantener una evolución continua del sistema y registrar cada una de las decisiones técnicas adoptadas durante el proceso de desarrollo.

El presente informe tiene como finalidad documentar íntegramente el desarrollo del Proyecto APT, describiendo la problemática abordada, la justificación de la solución propuesta, la metodología aplicada, el proceso de desarrollo, la arquitectura implementada, los resultados obtenidos y las evidencias que demuestran el cumplimiento de los objetivos establecidos. Asimismo, se presentan las principales decisiones técnicas adoptadas por el equipo y el impacto que la solución desarrollada puede generar dentro del contexto profesional para el cual fue concebida.

# Presentación del Proyecto

PointCheck corresponde a una plataforma tecnológica desarrollada para apoyar la administración integral de organizaciones cuya operación se basa en la prestación de servicios mediante reservas previamente agendadas. La solución fue concebida con el propósito de centralizar procesos que habitualmente son gestionados de manera independiente, permitiendo administrar usuarios, especialistas, reservas, atenciones, servicios, cobros y reportes desde un único sistema.

El proyecto fue desarrollado considerando una arquitectura cliente-servidor, donde una aplicación móvil Android interactúa permanentemente con un backend encargado de ejecutar la lógica de negocio y administrar la persistencia de los datos. Esta separación permite desacoplar completamente la interfaz de usuario de las reglas de negocio, favoreciendo la escalabilidad del sistema y facilitando futuras integraciones con otros clientes, como aplicaciones web o plataformas administrativas.

Desde el punto de vista funcional, PointCheck incorpora tres perfiles principales de usuario. El cliente dispone de herramientas para registrarse en la plataforma, buscar especialistas según distintas categorías de servicios, visualizar perfiles profesionales, reservar citas, consultar el estado de sus reservas y revisar su historial de atenciones. Además, para aquellos servicios que se realizan a domicilio, la plataforma integra información climática mediante OpenWeather API, permitiendo visualizar el pronóstico correspondiente al día de la atención.

El especialista cuenta con funcionalidades orientadas a la administración de su actividad profesional. Puede configurar su perfil público, administrar los servicios ofrecidos, establecer horarios laborales, gestionar reservas, registrar atenciones realizadas, visualizar indicadores de desempeño, generar reportes financieros y controlar la información relacionada con su actividad comercial.

Finalmente, el administrador dispone de herramientas de supervisión global que permiten gestionar usuarios registrados, controlar el estado general de la plataforma, acceder a registros de auditoría, administrar configuraciones generales del sistema y supervisar la información financiera generada por las distintas operaciones realizadas dentro de la aplicación.

Una característica relevante del proyecto corresponde a la evolución experimentada durante su desarrollo. A diferencia de desarrollar una aplicación completamente nueva, el equipo recibió inicialmente un prototipo funcional que sirvió como punto de partida para el trabajo. Sin embargo, el análisis técnico realizado durante las primeras etapas permitió identificar múltiples oportunidades de mejora relacionadas con la organización del código, la arquitectura utilizada y la incorporación de funcionalidades que no habían sido consideradas originalmente.

Como consecuencia de este análisis, se tomó la decisión de realizar una refactorización progresiva del proyecto, reorganizando completamente la estructura del backend y del frontend bajo una arquitectura Feature-Based. Esta decisión permitió aislar cada módulo funcional dentro de componentes independientes, facilitando el mantenimiento del sistema, reduciendo el acoplamiento entre funcionalidades y preparando la plataforma para futuras ampliaciones sin afectar el funcionamiento de los módulos ya existentes.

El desarrollo también contempló la incorporación de diversas mejoras orientadas a optimizar el rendimiento y fortalecer la seguridad del sistema. Entre ellas destacan la utilización de consultas SQL optimizadas para la generación de reportes, el uso de operaciones transaccionales para garantizar la consistencia de la información, la implementación de mecanismos de cifrado de contraseñas mediante BCrypt y la incorporación de auditoría para registrar acciones críticas ejecutadas por los distintos usuarios del sistema.

Como resultado del proceso de desarrollo, PointCheck evolucionó desde un prototipo con funcionalidades limitadas hacia una plataforma integral preparada para responder a necesidades reales de pequeñas y medianas empresas dedicadas a la prestación de servicios. La arquitectura implementada, junto con las tecnologías seleccionadas y las buenas prácticas aplicadas durante el desarrollo, permiten considerar la solución como una base sólida para futuras ampliaciones y nuevas líneas de desarrollo.



# Estructura del Informe

Con el propósito de facilitar la comprensión del trabajo realizado y mantener una organización coherente de la información, el presente informe ha sido estructurado siguiendo el ciclo completo de desarrollo del Proyecto APT.

Los primeros capítulos presentan la justificación del proyecto, los objetivos definidos y la metodología utilizada durante el desarrollo. Posteriormente se describe el proceso de planificación y ejecución del trabajo mediante Sprints, detallando la evolución del sistema desde el prototipo inicial hasta la versión final implementada.

En los capítulos centrales se documenta el desarrollo técnico del proyecto, incluyendo la arquitectura de software, el diseño de la base de datos, la implementación del backend, el desarrollo de la aplicación móvil, la integración entre componentes y la incorporación de mecanismos de seguridad y optimización del sistema.

Finalmente, el informe presenta los resultados obtenidos, el proceso de aseguramiento de calidad, las evidencias que respaldan el cumplimiento de los requerimientos comprometidos, las conclusiones alcanzadas por el equipo y las proyecciones futuras del proyecto.

# CAPÍTULO I. JUSTIFICACIÓN DEL PROYECTO

## 1. Justificación

La digitalización de los procesos empresariales se ha convertido en uno de los principales factores que determinan la competitividad de las organizaciones en la actualidad. La incorporación de tecnologías de la información ya no representa únicamente una ventaja competitiva, sino una necesidad para garantizar la eficiencia operativa, la correcta administración de recursos y una experiencia satisfactoria para los clientes. Sin embargo, a pesar del crecimiento sostenido de herramientas digitales, una gran cantidad de pequeñas y medianas empresas dedicadas a la prestación de servicios continúa administrando sus procesos mediante procedimientos manuales o soluciones tecnológicas aisladas que no permiten integrar completamente el funcionamiento del negocio.

Dentro de este contexto, negocios como barberías, peluquerías, centros de estética, consultas médicas, clínicas de rehabilitación, psicólogos, kinesiólogos, masoterapeutas y otros profesionales que trabajan mediante reservas continúan enfrentando dificultades relacionadas con la gestión de agendas, coordinación de horarios, seguimiento de clientes, administración de pagos y generación de reportes administrativos. En muchos casos estas actividades se realizan utilizando agendas físicas, hojas de cálculo o aplicaciones de mensajería instantánea, herramientas que fueron diseñadas para resolver necesidades puntuales, pero que no permiten mantener una administración integral de la información.

La utilización de estos mecanismos genera una serie de problemas operacionales que afectan directamente la productividad de los negocios. Entre ellos destacan la duplicidad de reservas, pérdida de información relevante, dificultad para administrar múltiples especialistas, inexistencia de historiales completos de atención, escasa trazabilidad de las operaciones realizadas y limitadas posibilidades para obtener indicadores que apoyen la toma de decisiones. Estas deficiencias impactan tanto en la organización interna como en la percepción de calidad por parte de los clientes, quienes actualmente esperan acceder a servicios digitales rápidos, seguros y disponibles desde dispositivos móviles.

Frente a esta problemática surge **PointCheck**, una plataforma integral desarrollada con el propósito de centralizar todos los procesos administrativos involucrados en la prestación de servicios mediante reservas. La solución busca integrar en un único sistema la administración de usuarios, especialistas, servicios, reservas, atenciones, cobros, reportes y configuraciones generales, permitiendo automatizar tareas repetitivas y disminuir considerablemente la posibilidad de errores asociados a la gestión manual de la información.

Una característica particular del proyecto consiste en que su desarrollo no comenzó desde una aplicación completamente nueva, sino a partir de un prototipo previamente existente que presentaba importantes limitaciones arquitectónicas y funcionales. Durante las primeras reuniones del equipo se realizó un análisis técnico del estado del software, identificando problemas relacionados con la organización del código, escasa modularidad, dificultades para incorporar nuevas funcionalidades y una arquitectura que comprometía la escalabilidad futura del sistema.

Como resultado de este análisis, el equipo tomó la decisión de realizar una refactorización completa del proyecto, reorganizando tanto el backend como la aplicación móvil bajo una arquitectura basada en funcionalidades (Feature-Based Architecture). Esta decisión permitió estructurar el software de manera más organizada, favoreciendo la reutilización del código, reduciendo el acoplamiento entre módulos y facilitando el mantenimiento del sistema conforme se incorporaban nuevas características.

El desarrollo de PointCheck también responde directamente al perfil profesional de un Ingeniero en Informática, ya que integra competencias relacionadas con análisis de requerimientos, diseño de arquitectura de software, desarrollo de aplicaciones móviles, implementación de servicios backend, modelamiento de bases de datos, integración mediante APIs REST, seguridad informática, aseguramiento de la calidad y gestión de proyectos utilizando metodologías ágiles. En consecuencia, el proyecto constituye una instancia de aplicación práctica de los conocimientos adquiridos durante la formación profesional, enfrentando problemáticas similares a las que pueden presentarse en organizaciones dedicadas al desarrollo de soluciones tecnológicas.

Desde el punto de vista tecnológico, PointCheck incorpora herramientas ampliamente utilizadas por la industria del software, entre ellas Kotlin, Jetpack Compose, Spring Boot, MySQL, Retrofit, Room, GitHub y metodologías ágiles basadas en Scrum. La utilización de estas tecnologías permitió desarrollar una solución moderna, mantenible y preparada para futuras ampliaciones, alineándose con las tendencias actuales del desarrollo de aplicaciones empresariales.

Asimismo, el proyecto incorpora mecanismos orientados a fortalecer la seguridad de la información siguiendo las recomendaciones de la norma ISO/IEC 27001. Se implementaron técnicas de cifrado de contraseñas utilizando BCrypt, control de acceso basado en roles, auditoría de acciones críticas, utilización de identificadores UUID para reducir riesgos asociados a ataques por enumeración y operaciones transaccionales mediante @Transactional, garantizando la integridad de los datos almacenados en el sistema.

En consecuencia, PointCheck no solamente constituye una solución informática funcional, sino que representa un proyecto técnicamente sólido, escalable y preparado para responder a necesidades reales dentro del mercado de prestación de servicios. Su desarrollo demuestra la capacidad del equipo para aplicar metodologías, herramientas y buenas prácticas propias de la Ingeniería en Informática, transformando

un prototipo inicial en una plataforma integral preparada para evolucionar continuamente mediante futuras mejoras.

# CAPÍTULO II. OBJETIVOS

## 2.1 Objetivo General

Diseñar, desarrollar e implementar una plataforma integral denominada **PointCheck**, orientada a la gestión de reservas, atención de clientes y administración de prestadores de servicios, mediante una arquitectura cliente-servidor moderna que permita integrar una aplicación móvil Android con un backend desarrollado en Spring Boot, proporcionando una solución escalable, segura y mantenible capaz de optimizar los procesos administrativos de pequeñas y medianas empresas dedicadas a la prestación de servicios.

## 2.2 Objetivos Específicos

Con el propósito de alcanzar el objetivo general planteado, se definieron los siguientes objetivos específicos que guiaron el desarrollo del proyecto durante cada uno de los Sprints ejecutados por el equipo.

### **Analizar el prototipo inicial e identificar oportunidades de mejora**

Realizar un levantamiento técnico del estado inicial del proyecto recibido, identificando limitaciones relacionadas con la arquitectura del software, organización del código, funcionalidades existentes, estructura de la base de datos y posibilidades de crecimiento futuro. Este análisis permitió establecer una hoja de ruta para la refactorización progresiva del sistema.

### **Diseñar una arquitectura escalable basada en módulos funcionales**

Reestructurar completamente la organización del proyecto implementando una arquitectura Feature-Based tanto en el backend como en la aplicación móvil, permitiendo desacoplar funcionalidades, facilitar el mantenimiento del software y favorecer la incorporación de nuevos módulos sin afectar el funcionamiento de los componentes existentes.

### **Implementar un backend robusto utilizando Spring Boot**

Desarrollar una API REST capaz de administrar la lógica de negocio del sistema, incorporando servicios para autenticación, gestión de usuarios, especialistas, reservas, servicios, atención de clientes, facturación, reportes y administración general de la plataforma, garantizando la correcta comunicación con la aplicación móvil.

## **Desarrollar una aplicación móvil moderna utilizando Kotlin y Jetpack Compose**

Diseñar una interfaz intuitiva y responsiva que permita a clientes, especialistas y administradores acceder a las funcionalidades del sistema mediante dispositivos Android, aplicando el patrón arquitectónico MVVM para mantener una adecuada separación entre la lógica de presentación y la lógica de negocio.

## **Diseñar una base de datos relacional eficiente**

Construir un modelo de datos capaz de representar correctamente todas las entidades involucradas en el funcionamiento del sistema, asegurando la integridad referencial, consistencia de la información y facilidad para futuras ampliaciones del proyecto.

## **Incorporar mecanismos de seguridad para la protección de la información**

Implementar medidas de seguridad inspiradas en la norma ISO/IEC 27001, incorporando cifrado de contraseñas mediante BCrypt, autenticación segura, control de acceso basado en roles, utilización de UUID, auditoría de acciones críticas y mecanismos transaccionales que garanticen la integridad de la información almacenada

## **Integrar servicios externos para enriquecer la experiencia del usuario**

Incorporar APIs externas que permitan complementar las funcionalidades del sistema, integrando OpenWeather para la consulta de condiciones climáticas asociadas a servicios realizados a domicilio y Google Maps para la geolocalización de especialistas y visualización de ubicaciones.

## **Optimizar el rendimiento y la experiencia de uso de la aplicación**

Implementar mecanismos de almacenamiento local mediante Room, optimizar consultas SQL para la generación de reportes, reducir tiempos de carga y mejorar la navegación entre pantallas, proporcionando una experiencia de usuario más fluida y eficiente.

## **Verificar la calidad del sistema mediante pruebas funcionales**

Diseñar y ejecutar casos de prueba orientados a validar el correcto funcionamiento de cada uno de los módulos implementados, identificando errores, aplicando correcciones y verificando el cumplimiento de los requerimientos comprometidos durante el desarrollo del proyecto.

## **Documentar íntegramente el proceso de desarrollo**

Registrar todas las actividades realizadas durante el proyecto, incluyendo planificación mediante Scrum, evolución de la arquitectura, decisiones técnicas, cambios implementados, pruebas realizadas y resultados obtenidos, generando una documentación que respalde el trabajo desarrollado y facilite la continuidad del proyecto en futuras etapas.

# CAPÍTULO III. ALCANCE DEL PROYECTO

## 3.1 Alcance General

El desarrollo de PointCheck tuvo como finalidad transformar un prototipo funcional en una plataforma integral orientada a la administración de negocios cuya operación depende de la reserva de servicios. Desde el inicio del proyecto se estableció que el objetivo principal no consistía únicamente en agregar nuevas funcionalidades, sino en realizar una evolución completa del sistema existente, mejorando su arquitectura, fortaleciendo la seguridad de la información y aumentando su capacidad de crecimiento a largo plazo.

El alcance definido para el Proyecto APT contempló el desarrollo de una solución cliente-servidor compuesta por una aplicación móvil Android y un backend desarrollado mediante Spring Boot, ambos integrados mediante servicios REST y apoyados por una base de datos relacional MySQL. La solución debía permitir administrar de forma centralizada el ciclo completo de atención de un servicio, comenzando desde el registro de usuarios hasta la generación de reportes administrativos y financieros.

A diferencia del prototipo inicial, cuyo funcionamiento estaba orientado principalmente a demostrar una idea conceptual, la versión desarrollada durante el Proyecto APT debía cumplir criterios propios de un software preparado para un entorno real de operación. Esto implicó reorganizar completamente la estructura del código, implementar una arquitectura escalable, mejorar la experiencia de usuario, fortalecer los mecanismos de seguridad y garantizar la consistencia de los datos mediante buenas prácticas de desarrollo.

Durante las primeras etapas del proyecto se realizó un análisis exhaustivo del estado inicial de la aplicación, identificando limitaciones relacionadas con la organización del código, duplicidad de responsabilidades entre componentes, escasa separación de capas y dificultades para incorporar nuevas funcionalidades sin afectar módulos existentes. Como consecuencia de este análisis, el equipo definió una hoja de ruta basada en refactorizaciones progresivas que permitieron evolucionar el sistema sin perder continuidad en el desarrollo.

El alcance también contempló la incorporación de funcionalidades diferenciadas según el tipo de usuario, estableciendo tres perfiles principales dentro del sistema: clientes, especialistas y administradores. Cada uno de ellos dispone de herramientas específicas que responden a sus necesidades operacionales, evitando concentrar todas las funcionalidades dentro de una única interfaz y favoreciendo una experiencia de uso más intuitiva.



## 3.2 Funcionalidades comprometidas

Como resultado del análisis inicial y de las reuniones de planificación realizadas durante los primeros Sprints, se estableció un conjunto de funcionalidades consideradas indispensables para la versión final del proyecto.

Para el perfil **Ciente**, el sistema incorpora un proceso completo de registro e inicio de sesión, navegación por categorías de servicios, búsqueda de especialistas según distintos criterios, visualización de perfiles profesionales, reserva de citas mediante selección de fecha y horario, consulta del estado de las reservas realizadas y acceso al historial completo de atenciones. Asimismo, cuando el servicio corresponde a una atención a domicilio, la aplicación permite visualizar el pronóstico climático asociado a la fecha seleccionada mediante la integración con OpenWeather API.

El perfil **Especialista** dispone de herramientas orientadas a la administración de su actividad profesional. Entre ellas se encuentran la configuración del perfil comercial, administración del catálogo de servicios ofrecidos, definición de horarios laborales, control de la agenda de reservas, registro del inicio y término de las atenciones, seguimiento de ingresos generados, consulta de indicadores de desempeño y generación de reportes financieros exportables.

Por otra parte, el perfil **Administrador** incorpora funcionalidades destinadas al control global de la plataforma. Estas herramientas permiten administrar usuarios registrados, habilitar o bloquear cuentas, consultar registros de auditoría, supervisar el comportamiento general del sistema, administrar configuraciones globales y controlar la información financiera consolidada de la plataforma.

Además de estas funcionalidades principales, el proyecto contempla mecanismos complementarios que enriquecen la experiencia de usuario y fortalecen el funcionamiento del sistema, tales como almacenamiento local mediante Room para reducir tiempos de carga, utilización de UUID como identificadores únicos, consultas SQL optimizadas para la generación de reportes y un sistema de auditoría que registra acciones críticas realizadas dentro de la plataforma.

## 3.3 Funcionalidades fuera del alcance

Como ocurre en todo proyecto de desarrollo de software, durante la planificación fue necesario establecer límites claros respecto de las funcionalidades que serían implementadas durante el período académico.

Aunque la arquitectura desarrollada permite incorporar futuras ampliaciones, algunas características fueron deliberadamente excluidas del alcance de esta versión para concentrar los esfuerzos del equipo en garantizar la estabilidad y calidad de las funcionalidades principales.

Entre las mejoras consideradas para futuras versiones se encuentran la implementación de notificaciones Push mediante Firebase Cloud Messaging, integración con pasarelas de pago para realizar cobros directamente desde la aplicación, autenticación mediante proveedores externos como Google o Microsoft, versión web administrativa desarrollada con tecnologías frontend dedicadas, soporte para múltiples idiomas, sistema de evaluación pública de especialistas y herramientas avanzadas de analítica basadas en inteligencia artificial.

Estas funcionalidades representan oportunidades de evolución del proyecto, pero no fueron consideradas indispensables para cumplir los objetivos definidos para el Proyecto APT.

### **3.4 Beneficiarios del proyecto**

PointCheck fue concebido como una solución adaptable a distintos tipos de organizaciones que prestan servicios mediante reserva previa. Entre los principales beneficiarios se encuentran barberías, centros de estética, peluquerías, consultas médicas, clínicas odontológicas, centros de rehabilitación, psicólogos, kinesiólogos, nutricionistas, entrenadores personales y profesionales independientes que requieren administrar su agenda de manera eficiente.

La plataforma también beneficia directamente a los clientes finales, quienes disponen de una herramienta moderna para consultar especialistas, reservar servicios, revisar su historial de atenciones y gestionar sus citas desde dispositivos móviles sin depender de procesos manuales o comunicación telefónica.

Finalmente, la arquitectura implementada convierte a PointCheck en una base tecnológica preparada para continuar evolucionando, permitiendo incorporar nuevos módulos funcionales sin afectar la estabilidad del sistema existente.

# CAPÍTULO IV. METODOLOGÍA DE DESARROLLO

## 4.1 Selección de la metodología

El desarrollo de PointCheck fue gestionado utilizando la metodología ágil **Scrum**, debido a que sus principios se adaptan adecuadamente a proyectos de desarrollo de software caracterizados por una evolución continua de los requerimientos y la incorporación progresiva de nuevas funcionalidades.

La elección de Scrum respondió principalmente a la necesidad de organizar el trabajo del equipo de manera iterativa, permitiendo evaluar constantemente el avance del proyecto y realizar ajustes conforme se identificaban nuevas necesidades durante el proceso de refactorización del prototipo inicial. Considerando que gran parte del trabajo consistía en mejorar una aplicación previamente desarrollada, resultaba fundamental contar con una metodología que facilitara la planificación incremental y la incorporación gradual de mejoras sin comprometer la estabilidad del sistema.

Otro aspecto relevante corresponde a la posibilidad de mantener una comunicación permanente entre los integrantes del equipo. Las reuniones periódicas permitieron revisar el estado del proyecto, coordinar el trabajo de backend, frontend y documentación, identificar bloqueos técnicos y establecer prioridades para los siguientes períodos de desarrollo.

La naturaleza iterativa de Scrum también favoreció la validación continua del software, permitiendo detectar errores tempranamente y reducir el impacto de cambios arquitectónicos realizados durante el proyecto.

## 4.2 Organización del equipo

El proyecto fue desarrollado por un equipo compuesto por tres integrantes, quienes distribuyeron las responsabilidades considerando sus fortalezas técnicas y manteniendo una colaboración constante durante todo el desarrollo.

Si bien cada integrante asumió responsabilidades principales dentro de determinadas áreas, todas las decisiones relevantes fueron discutidas colectivamente durante las reuniones de planificación y revisión realizadas al término de cada Sprint. Esta dinámica permitió mantener una visión compartida del proyecto y asegurar la integración permanente entre frontend, backend y documentación.

Las actividades relacionadas con el desarrollo del backend se concentraron principalmente en la implementación de servicios REST, lógica de negocio, persistencia de datos y mecanismos de seguridad. Paralelamente, el desarrollo del frontend estuvo

orientado a la construcción de interfaces utilizando Jetpack Compose, navegación entre pantallas, integración con ViewModels y optimización de la experiencia de usuario. Finalmente, la documentación técnica evolucionó de forma paralela al desarrollo del software, registrando decisiones arquitectónicas, planificación, avances obtenidos y resultados de las pruebas realizadas.

El uso de GitHub permitió coordinar el trabajo colaborativo mediante ramas independientes destinadas al desarrollo de nuevas funcionalidades, pruebas de interfaz y refactorizaciones estructurales antes de integrar los cambios a la rama principal del proyecto.

## 4.3 Desarrollo mediante Sprints

La planificación del proyecto se estructuró mediante **Sprints de dos semanas**, permitiendo organizar el trabajo en ciclos cortos de desarrollo con objetivos claramente definidos.

Cada Sprint comenzó con una reunión de planificación donde el equipo analizaba el estado actual del proyecto, revisaba los avances obtenidos durante el período anterior y definía las actividades prioritarias para las siguientes dos semanas. Estas reuniones también permitían identificar riesgos técnicos, redistribuir tareas cuando era necesario y establecer criterios de aceptación para las funcionalidades comprometidas.

Durante la ejecución de cada Sprint se desarrollaban simultáneamente actividades relacionadas con backend, frontend, pruebas y documentación, procurando mantener una evolución equilibrada del sistema. Al finalizar cada iteración se realizaba una revisión general del trabajo desarrollado, validando el funcionamiento de las nuevas funcionalidades antes de incorporarlas al flujo principal de desarrollo.

Esta estrategia permitió mantener un crecimiento progresivo del proyecto, reduciendo considerablemente la aparición de conflictos entre componentes y facilitando la incorporación de mejoras arquitectónicas sin interrumpir el desarrollo de nuevas funcionalidades.

La planificación mediante Sprints también permitió documentar detalladamente la evolución del proyecto, registrando cada modificación importante realizada desde el análisis del prototipo inicial hasta la implementación de la versión final de PointCheck.

## **4.4 Herramientas utilizadas durante la gestión del proyecto**

Como complemento a la metodología Scrum, el equipo utilizó diversas herramientas que facilitaron la coordinación del trabajo y el control del desarrollo.

GitHub fue empleado como plataforma principal para el control de versiones, permitiendo mantener un historial completo de cambios, administrar ramas de desarrollo y facilitar la integración progresiva de nuevas funcionalidades.

IntelliJ IDEA y Android Studio fueron utilizados como entornos de desarrollo para backend y frontend respectivamente, mientras que MySQL Workbench permitió administrar el modelo de datos y validar consultas durante el desarrollo del sistema.

La documentación técnica evolucionó de manera paralela al software, incorporando diagramas, planificación de actividades, informes de avance, casos de prueba y registros de las decisiones técnicas adoptadas por el equipo durante cada etapa del proyecto.

# CAPÍTULO V. PLANIFICACIÓN Y DESARROLLO DEL PROYECTO

## 5.1 Planificación General

El desarrollo de PointCheck fue organizado mediante una planificación incremental basada en la metodología Scrum, estructurando el trabajo en Sprints de dos semanas de duración. Esta estrategia permitió dividir un proyecto de gran complejidad en ciclos de desarrollo más pequeños y controlables, facilitando el seguimiento del avance, la identificación temprana de problemas y la incorporación progresiva de nuevas funcionalidades.

A diferencia de un proyecto desarrollado completamente desde cero, PointCheck comenzó a partir de un prototipo previamente implementado. Durante las primeras reuniones del equipo se concluyó que la estructura existente no permitía sostener el crecimiento esperado del sistema, principalmente debido a problemas relacionados con la organización del código, la escasa separación entre responsabilidades y la dificultad para incorporar nuevas funcionalidades sin afectar componentes existentes.

Por esta razón, antes de iniciar el desarrollo de nuevas características se decidió establecer una planificación orientada inicialmente a comprender el funcionamiento del prototipo, identificar oportunidades de mejora y definir una arquitectura que permitiera construir una solución más robusta y escalable.

Cada Sprint contempló actividades de planificación, desarrollo, integración, pruebas y documentación. Al finalizar cada iteración se realizó una revisión del trabajo desarrollado, evaluando el cumplimiento de los objetivos establecidos y definiendo las prioridades correspondientes al siguiente ciclo de trabajo.

Esta planificación permitió mantener una evolución continua del proyecto, registrando mediante GitHub cada una de las modificaciones realizadas sobre el sistema y facilitando el trabajo colaborativo mediante el uso de ramas independientes para el desarrollo de nuevas funcionalidades.

## 5.2 Evolución del proyecto mediante Sprints

### Sprint 1

#### Análisis del prototipo y definición de la arquitectura

**Período:** 30 de marzo de 2026 – 10 de abril de 2026

El primer Sprint estuvo enfocado principalmente en comprender el funcionamiento del prototipo recibido y establecer las bases técnicas sobre las cuales se desarrollaría el resto del proyecto.

Durante este período se realizaron reuniones de análisis funcional donde el equipo identificó las principales limitaciones presentes en la aplicación. Entre ellas destacaban una organización poco estructurada del código, dificultades para mantener una adecuada separación entre la lógica de negocio y la interfaz de usuario, duplicidad de responsabilidades entre componentes y una arquitectura que complicaba considerablemente la incorporación de nuevas funcionalidades.

Como resultado de este análisis se decidió adoptar una **Feature-Based Architecture**, reorganizando completamente la estructura tanto del backend como de la aplicación móvil. Esta decisión permitiría agrupar controladores, servicios, repositorios, modelos, ViewModels e interfaces de usuario según la funcionalidad que representan, facilitando el mantenimiento futuro del sistema.

Durante este Sprint también se definieron las tecnologías que formarían parte de la solución definitiva, seleccionando Kotlin y Jetpack Compose para el desarrollo móvil, Spring Boot para el backend, MySQL como sistema gestor de bases de datos y GitHub como plataforma para el control de versiones.

Paralelamente comenzó la reorganización inicial del repositorio, definiéndose una estrategia de trabajo basada en ramas independientes destinadas al desarrollo de nuevas funcionalidades, pruebas de interfaz y procesos de refactorización antes de integrar los cambios al proyecto principal.

Como resultado de este Sprint quedó establecida la arquitectura general del sistema, el plan de trabajo inicial y la estrategia de desarrollo que se utilizaría durante el resto del proyecto.

## Sprint 2

### Refactorización estructural y reorganización del proyecto

**Período:** 13 de abril de 2026 – 24 de abril de 2026

Con la arquitectura definida durante el Sprint anterior, el segundo ciclo de desarrollo estuvo dedicado principalmente a iniciar la refactorización completa del proyecto.

Las actividades se concentraron en reorganizar la estructura del backend bajo un enfoque modular, trasladando controladores, servicios, repositorios y modelos hacia módulos independientes organizados por funcionalidades.

En paralelo comenzó la reorganización de la aplicación Android utilizando el patrón MVVM, separando correctamente las capas de presentación, lógica de negocio y acceso a datos. Esta modificación permitió reducir significativamente el acoplamiento existente entre las pantallas y preparar el proyecto para la incorporación de nuevas características.

Durante este Sprint también se inició la actualización de los modelos de datos utilizados tanto por el backend como por la aplicación móvil, adaptándolos a la nueva estructura del sistema y preparando la futura migración hacia identificadores UUID.

Otra actividad importante correspondió a la creación de ramas especializadas destinadas a realizar pruebas de interfaz de usuario antes de integrar los cambios al desarrollo principal. Esta estrategia permitió experimentar con nuevos diseños sin afectar la estabilidad del proyecto.

Al finalizar el Sprint, el sistema mantenía su funcionamiento básico, pero ya contaba con una estructura considerablemente más organizada y preparada para continuar evolucionando.



# Sprint 3

## Implementación de funcionalidades principales

**Período:** 27 de abril de 2026 – 8 de mayo de 2026

Una vez finalizada gran parte de la reorganización arquitectónica, el tercer Sprint estuvo orientado a desarrollar las primeras funcionalidades de negocio sobre la nueva estructura.

Durante este período se fortalecieron los módulos relacionados con autenticación, administración de usuarios y gestión de especialistas, estableciendo las bases para implementar posteriormente los distintos roles del sistema.

En el backend comenzaron a consolidarse los servicios REST encargados de administrar reservas, perfiles profesionales y servicios ofrecidos por cada especialista. Paralelamente, la aplicación Android incorporó nuevas pantallas desarrolladas completamente mediante Jetpack Compose, siguiendo criterios modernos de diseño y reutilización de componentes.

También comenzaron las primeras integraciones entre frontend y backend utilizando Retrofit, permitiendo validar el flujo completo de información entre ambas capas del sistema.

A nivel documental se actualizó la planificación general del proyecto y se registraron las primeras decisiones arquitectónicas adoptadas por el equipo, generando evidencia del proceso de evolución del software.

Como resultado de este Sprint, PointCheck dejó de ser únicamente una reorganización del prototipo para transformarse progresivamente en una plataforma funcional basada en una arquitectura completamente nueva.

# Sprint 4

## Consolidación de reservas y experiencia de usuario

**Período:** 11 de mayo de 2026 – 22 de mayo de 2026

Durante el cuarto Sprint el esfuerzo del equipo se concentró en consolidar el flujo principal del negocio: el proceso completo de reservas.

Se desarrolló el motor de agendamiento permitiendo controlar disponibilidad horaria, registrar nuevas reservas y administrar correctamente el historial de citas tanto para clientes como para especialistas.

Simultáneamente se mejoró la navegación general de la aplicación, reorganizando pantallas, optimizando la comunicación entre ViewModels y reduciendo la complejidad de distintas rutas internas.

En el backend se continuó fortaleciendo la lógica de negocio mediante validaciones adicionales y mejoras en la persistencia de la información, preparando el sistema para soportar funcionalidades más complejas durante los siguientes Sprints.

Como complemento, comenzaron las primeras pruebas funcionales destinadas a verificar el correcto comportamiento del proceso completo de reservas.

## 5.3 Evolución continua del proyecto

Los Sprints posteriores mantuvieron la misma dinámica de trabajo incremental, incorporando progresivamente nuevas funcionalidades como la gestión de atenciones, facturación, dashboards, reportes financieros, integración con APIs externas, optimización del rendimiento, almacenamiento local mediante Room, implementación de mecanismos avanzados de seguridad, auditoría de acciones críticas y mejoras generales en la experiencia de usuario.

Cada iteración permitió incrementar la madurez del sistema, reducir deuda técnica identificada durante los ciclos anteriores y fortalecer la integración entre frontend y backend. Paralelamente, la documentación evolucionó junto con el software, registrando decisiones de diseño, resultados obtenidos, pruebas ejecutadas y evidencias del avance del proyecto.

Como resultado de este proceso iterativo, PointCheck evolucionó desde un prototipo con funcionalidades limitadas hacia una plataforma integral, escalable y preparada para responder a necesidades reales de pequeñas y medianas empresas dedicadas a la prestación de servicios.

## 5.4 Resultados de la planificación

La utilización de una planificación basada en Scrum permitió mantener un control permanente sobre el avance del proyecto, favoreciendo la coordinación entre los integrantes del equipo y facilitando la incorporación progresiva de mejoras sin comprometer la estabilidad del sistema.

El uso de Sprints también permitió mantener una trazabilidad completa de la evolución del proyecto mediante el historial de commits registrado en GitHub, evidenciando el trabajo continuo realizado desde abril de 2026 hasta la obtención de la versión final de PointCheck.

Al finalizar el proceso de desarrollo, todos los objetivos definidos durante la planificación fueron alcanzados, logrando implementar una solución considerablemente más robusta que el prototipo inicial y cumpliendo los requerimientos establecidos para el Proyecto APT.

# CAPÍTULO VI. DESARROLLO TÉCNICO DEL PROYECTO

## 6.1 Arquitectura General del Sistema

El desarrollo de PointCheck fue concebido bajo una arquitectura cliente-servidor, separando completamente la aplicación móvil de la lógica de negocio implementada en el backend. Esta decisión permitió desacoplar las responsabilidades del sistema, facilitando el mantenimiento, la escalabilidad y la incorporación de nuevas plataformas cliente sin necesidad de modificar la lógica central de la aplicación.

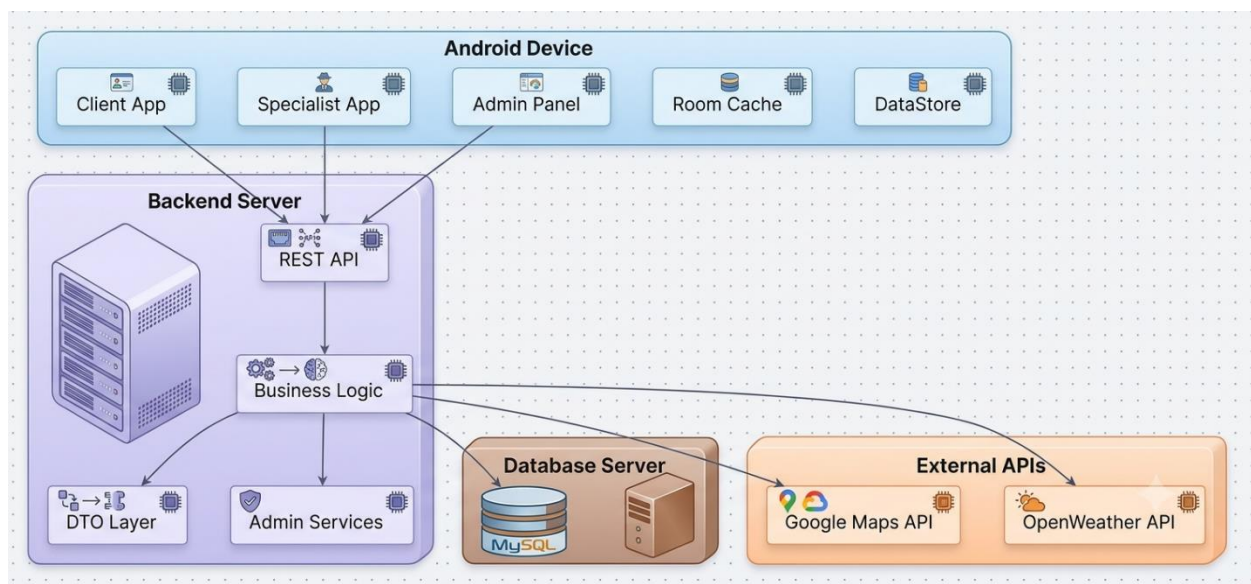
Desde el punto de vista arquitectónico, la solución se encuentra compuesta por cuatro capas principales: la aplicación móvil Android, el backend desarrollado con Spring Boot, la base de datos MySQL y un conjunto de servicios externos utilizados para complementar determinadas funcionalidades del sistema.

La aplicación móvil constituye el punto de interacción con los usuarios, proporcionando interfaces diferenciadas según el rol de cada perfil. Todas las operaciones realizadas por los usuarios son enviadas mediante solicitudes HTTP hacia el backend utilizando servicios REST implementados sobre el protocolo HTTP.

El backend actúa como núcleo del sistema, siendo responsable de ejecutar toda la lógica de negocio, validar la información recibida, administrar la autenticación, aplicar reglas de seguridad y coordinar el acceso a la base de datos.

Finalmente, la persistencia de la información es gestionada mediante MySQL utilizando Spring Data JPA como mecanismo de acceso a los datos, garantizando integridad referencial, consistencia transaccional y facilidad para futuras ampliaciones del modelo.

Esta arquitectura proporciona una clara separación entre presentación, negocio y persistencia, permitiendo que cada componente evolucione de manera independiente sin afectar el funcionamiento global del sistema.



## 6.2 Evolución de la Arquitectura

Uno de los cambios más importantes realizados durante el Proyecto APT fue la transformación completa de la arquitectura originalmente utilizada por el prototipo.

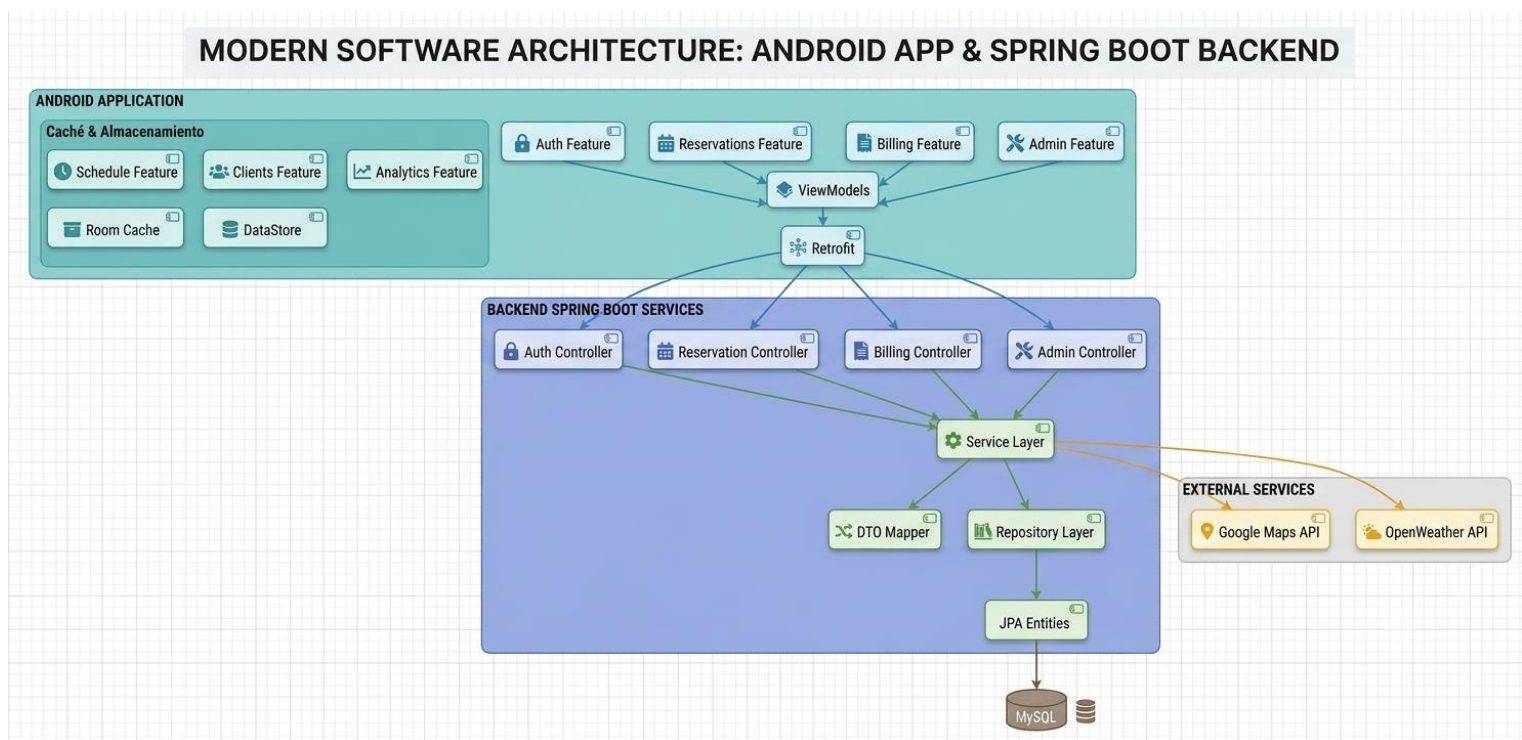
El análisis realizado durante los primeros Sprints permitió identificar que la estructura inicial presentaba un alto nivel de acoplamiento entre componentes, dificultando considerablemente la incorporación de nuevas funcionalidades y aumentando el riesgo de introducir errores durante el mantenimiento del software.

Frente a esta situación, el equipo decidió migrar progresivamente hacia una **Feature-Based Architecture**, reorganizando tanto el backend como la aplicación móvil.

En lugar de estructurar el proyecto según capas tradicionales, donde todos los controladores, servicios y repositorios se encontraban agrupados por tipo de componente, la nueva arquitectura organiza el sistema según funcionalidades.

De esta forma, cada módulo concentra todos los elementos necesarios para cumplir una responsabilidad específica.

Por ejemplo, el módulo **Reservations** contiene su controlador, servicio, repositorio, entidades, DTOs y clases auxiliares, mientras que módulos como **Billing**, **Reports**, **Authentication** o **Professional Profile** mantienen la misma organización independiente.



Esta decisión produjo múltiples beneficios durante el desarrollo.

En primer lugar, permitió reducir considerablemente el acoplamiento entre funcionalidades.

En segundo lugar, facilitó el trabajo colaborativo del equipo, ya que distintos integrantes podían trabajar simultáneamente sobre módulos diferentes sin generar conflictos constantes.

Finalmente, esta organización prepara la plataforma para incorporar futuras funcionalidades sin modificar componentes ya existentes.

## 6.3 Desarrollo del Backend

El backend fue implementado utilizando **Spring Boot**, seleccionado por su robustez, facilidad para construir APIs REST y amplia utilización dentro del desarrollo empresarial moderno.

Toda la lógica de negocio del sistema se encuentra centralizada en esta capa, siendo responsable de validar operaciones, controlar permisos de acceso, administrar reservas, generar reportes, procesar cobros y garantizar la consistencia de la información almacenada.

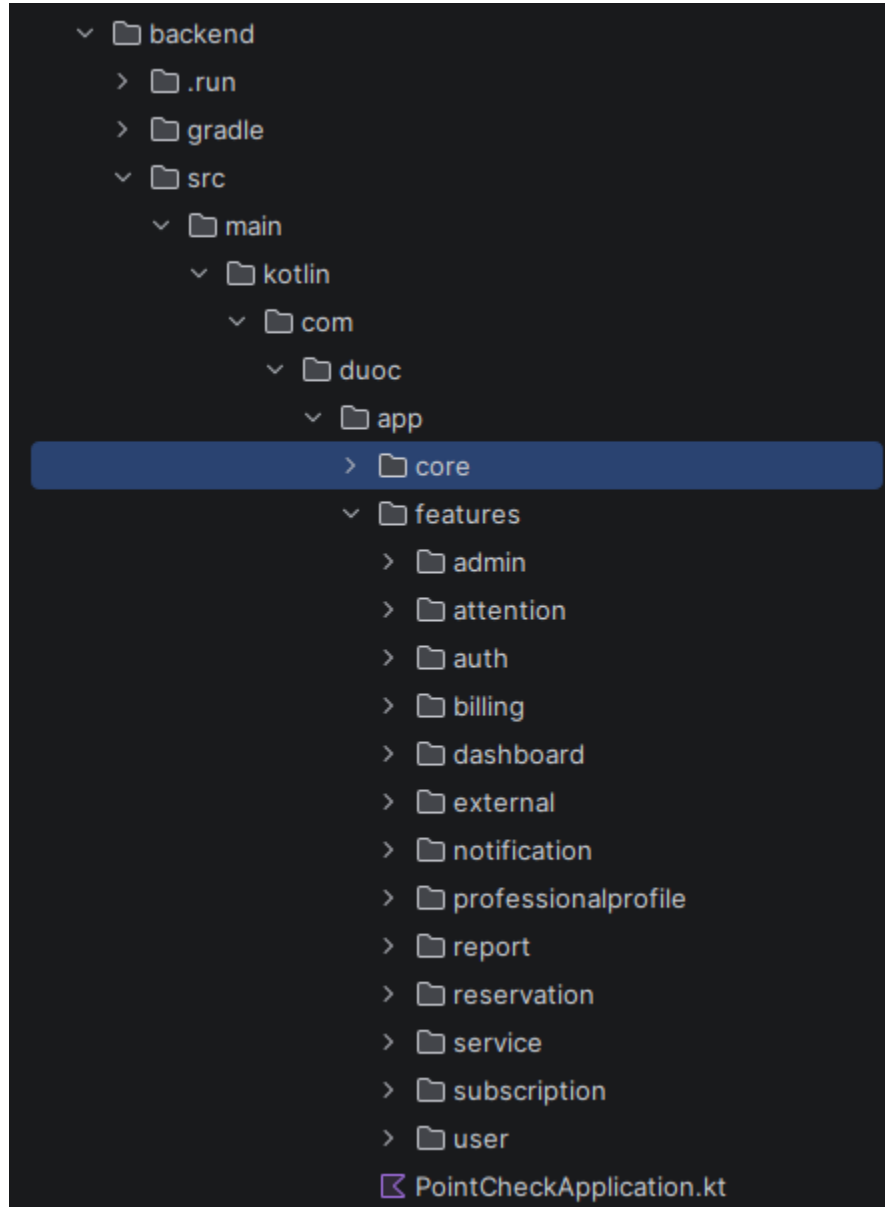
La estructura del backend fue reorganizada completamente durante el proceso de refactorización, adoptando una arquitectura modular basada en funcionalidades.

Cada módulo implementa una estructura interna compuesta por:

- Controller
- Service
- Repository
- DTO
- Entity
- Mapper
- Validation

Esta organización facilita la mantenibilidad del código y permite que cada funcionalidad evolucione de forma independiente.

Entre los principales módulos implementados destacan:



Cada uno de estos módulos encapsula completamente la lógica correspondiente a su dominio funcional.



## 6.4 Lógica de Negocio

La lógica de negocio constituye el núcleo del sistema y fue desarrollada siguiendo principios de separación de responsabilidades.

Los controladores REST únicamente reciben las solicitudes HTTP provenientes de la aplicación móvil y delegan completamente el procesamiento hacia la capa de servicios.

Los servicios contienen todas las reglas del negocio.

Entre ellas destacan:

- validación de disponibilidad horaria;
- verificación de conflictos entre reservas;
- cálculo de ingresos;
- generación automática de registros de cobro;
- administración de estados de atención;
- control de usuarios activos;
- validación de permisos según rol.

Esta separación permitió desarrollar un backend considerablemente más limpio, reutilizable y fácil de mantener.

## 6.5 Desarrollo del Frontend

La aplicación móvil fue desarrollada utilizando **Kotlin** y **Jetpack Compose**, tecnologías modernas recomendadas oficialmente para el desarrollo Android.

Desde las primeras etapas del proyecto se decidió abandonar el enfoque tradicional basado en XML y adoptar Compose debido a las ventajas que ofrece respecto al desarrollo declarativo de interfaces.

Toda la aplicación fue organizada bajo el patrón arquitectónico **MVVM (Model-View-ViewModel)**.

Esta arquitectura separa claramente tres responsabilidades fundamentales:

La interfaz de usuario (View), encargada únicamente de representar información.

Los ViewModels, responsables de administrar el estado de la aplicación y coordinar la comunicación con los repositorios.

Los repositorios, encargados de interactuar con el backend mediante Retrofit.

Gracias a esta separación fue posible reducir considerablemente el código duplicado y facilitar las pruebas de cada componente.

## 6.6 Organización del Frontend

Al igual que el backend, la aplicación Android fue reorganizada completamente utilizando una arquitectura basada en funcionalidades.

Cada módulo contiene todos los elementos necesarios para implementar una característica específica.

Por ejemplo:

auth/  
reservations/  
billing/  
dashboard/  
reports/  
professional\_profile/  
services/  
categories/

Dentro de cada módulo se encuentran:

- Screens
- ViewModels
- Repository
- UI Components
- State
- Events

Esta organización facilita el mantenimiento del proyecto y permite agregar nuevas pantallas sin afectar módulos existentes.

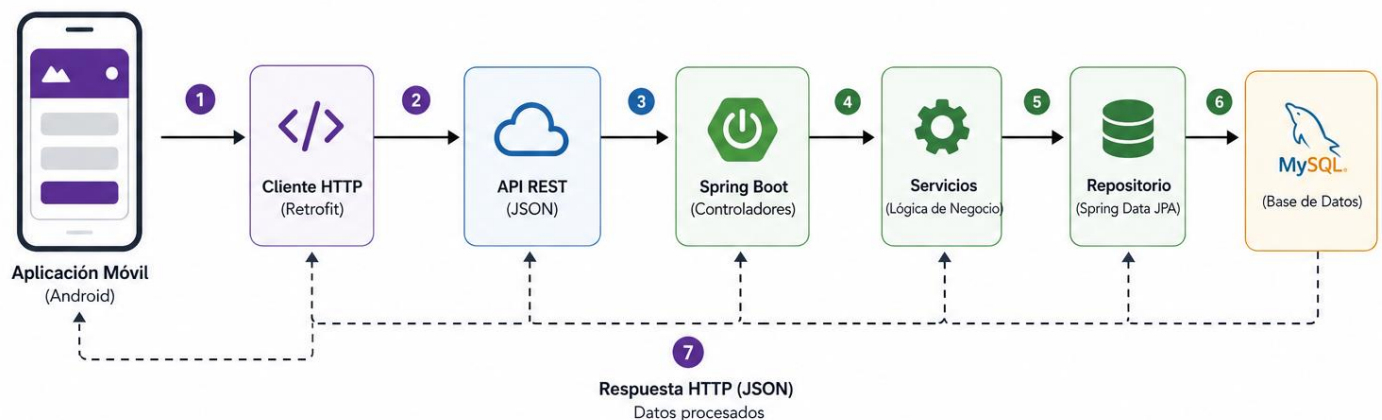
## 6.7 Comunicación entre Frontend y Backend

La comunicación entre ambas capas fue implementada utilizando una arquitectura REST.

1. El usuario interactúa con la interfaz.
2. El ViewModel procesa la acción.
3. El Repository realiza la llamada HTTP.
4. Retrofit serializa los datos en formato JSON.
5. Spring Boot recibe la solicitud.
6. El Controller delega la lógica al Service.
7. El Service consulta el Repository.
8. JPA interactúa con MySQL.
9. La respuesta vuelve al cliente.
10. El ViewModel actualiza automáticamente la interfaz mediante StateFlow.

**Figura 6.7 – Comunicación Frontend ↔ Backend**

Flujo general de comunicación entre la aplicación móvil (Android) y el backend (Spring Boot)



**Descripción del flujo:**

- 1 El usuario realiza una acción en la aplicación móvil.
- 2 La aplicación utiliza Retrofit para enviar la solicitud HTTP.
- 3 La solicitud llega a la API REST del backend en formato JSON.
- 4 El controlador recibe la petición y la delega a los servicios correspondientes.
- 5 Los servicios ejecutan la lógica de negocio.
- 6 El repositorio accede a la base de datos MySQL para consultar o guardar información.
- 7 La respuesta viaja de vuelta al cliente en formato JSON y se muestra en la aplicación.

**Leyenda**

- Flujo de solicitud
- Flujo de respuesta

Este flujo garantiza una comunicación completamente desacoplada entre ambas capas.

## 6.8 Base de Datos

La persistencia de la información fue implementada utilizando **MySQL**, permitiendo almacenar toda la información relacionada con usuarios, especialistas, reservas, servicios, cobros, auditorías y reportes.

Durante el desarrollo del proyecto el modelo de datos evolucionó considerablemente respecto del prototipo inicial.

Se normalizaron relaciones.

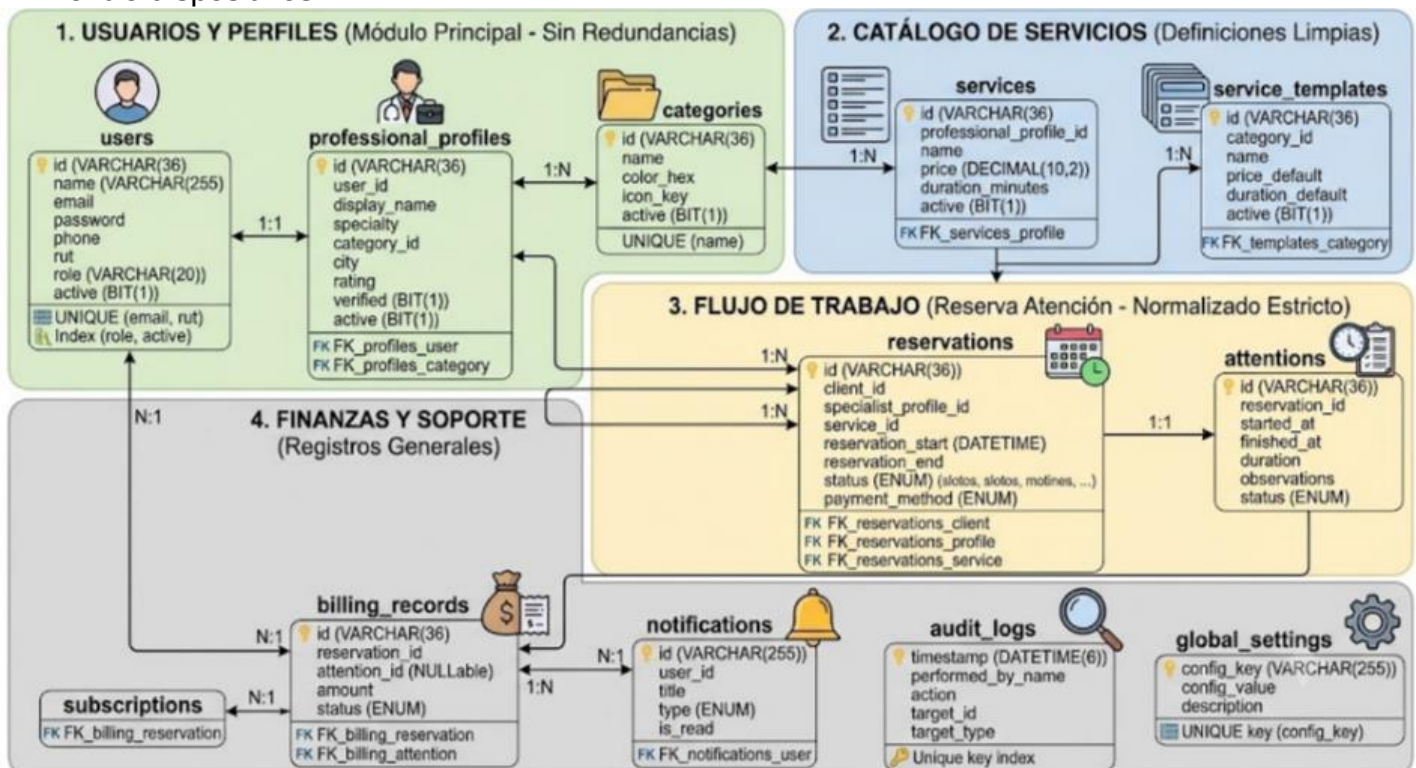
Se eliminaron redundancias.

Se mejoraron claves foráneas.

Se incorporaron nuevas entidades.

Se optimizaron índices.

Además, el sistema migró completamente desde identificadores numéricos hacia **UUID**, incrementando significativamente la seguridad y facilitando futuras sincronizaciones entre dispositivos.



Cada entidad representa un dominio específico dentro del funcionamiento de PointCheck.

## 6.9 APIs Externas

Con el propósito de enriquecer la experiencia del usuario, el sistema integra distintos servicios externos.

### **OpenWeather API**

Permite consultar las condiciones climáticas asociadas a servicios realizados a domicilio.

Esta funcionalidad resulta especialmente útil para profesionales cuya actividad depende de factores climáticos

### **Google Maps API**

Se utiliza para almacenar y visualizar la ubicación geográfica de especialistas.

Además facilita futuras implementaciones relacionadas con cálculo de distancias y navegación.

## 6.10 Seguridad del Sistema

Uno de los aspectos que experimentó mayor evolución durante el Proyecto APT fue la seguridad.

Las mejoras implementadas fueron diseñadas siguiendo las recomendaciones establecidas por la norma **ISO/IEC 27001**, incorporando controles orientados a proteger la confidencialidad, integridad y disponibilidad de la información.

Entre las principales medidas implementadas destacan:

### **BCrypt**

Las contraseñas dejaron de almacenarse como texto plano.

Antes de persistirse en la base de datos son cifradas utilizando BCrypt.

Durante el proceso de autenticación se utiliza `passwordEncoder.matches()`, evitando comparar contraseñas directamente.

Esto impide recuperar las credenciales originales incluso en caso de una filtración de la base de datos.

### **Control de acceso basado en roles**

El sistema incorpora tres perfiles claramente diferenciados:

- CLIENT
- SPECIALIST
- ADMIN

Cada uno posee permisos específicos sobre los recursos disponibles.

Esta estrategia reduce considerablemente la superficie de ataque y evita accesos no autorizados.

### **UUID**

La migración hacia UUID elimina el riesgo asociado a la enumeración de identificadores consecutivos.

Esto dificulta ataques donde un usuario intenta acceder a recursos modificando manualmente identificadores numéricos.

### **Auditoría**

Todas las operaciones críticas quedan registradas en la entidad **Audit Logs**.

Cada registro almacena:

- usuario
- fecha
- acción realizada
- recurso afectado

Esta funcionalidad proporciona trazabilidad completa del sistema.

## **Transacciones**

Las operaciones críticas utilizan @Transactional.

Esto garantiza que múltiples operaciones relacionadas sean ejecutadas completamente o revertidas automáticamente en caso de producirse algún error.

## **Optimización SQL**

Los reportes dejaron de generarse utilizando filtros sobre colecciones en memoria.

Las consultas fueron trasladadas directamente hacia MySQL mediante JPQL y consultas nativas.

Como resultado se redujo considerablemente el consumo de memoria y el tiempo necesario para generar indicadores.



## 6.11 Cumplimiento de la ISO/IEC 27001

Aunque PointCheck no fue certificado formalmente bajo la norma ISO/IEC 27001, durante su desarrollo se adoptaron múltiples controles alineados con este estándar internacional de seguridad de la información. Estas prácticas fueron incorporadas desde el diseño de la arquitectura con el objetivo de fortalecer la protección de los datos, minimizar riesgos operacionales y asegurar un tratamiento responsable de la información gestionada por la plataforma.

Entre los controles aplicados destacan la autenticación segura mediante contraseñas cifradas con BCrypt, la segregación de funciones mediante control de acceso basado en roles, la implementación de registros de auditoría para garantizar la trazabilidad de acciones críticas, el uso de identificadores UUID para reducir riesgos asociados a la enumeración de recursos, la aplicación de transacciones para preservar la integridad de la información y la optimización de consultas que disminuyen la exposición innecesaria de datos en memoria.

Asimismo, la organización modular del sistema facilita la aplicación del principio de mínimo privilegio, el mantenimiento de componentes de forma independiente y la futura incorporación de controles adicionales como autenticación multifactor, cifrado de datos sensibles en reposo y monitoreo avanzado de eventos de seguridad.

La adopción de estas prácticas demuestra que el desarrollo del proyecto consideró la seguridad como un proceso transversal y no como una característica incorporada únicamente al finalizar la implementación, alineándose con los principios de *Security by Design* promovidos por la industria del desarrollo de software.

# CAPÍTULO VII. VERIFICACIÓN DE LA CALIDAD DEL SISTEMA

## 7.1 Aseguramiento de la Calidad

La calidad del software constituye uno de los pilares fundamentales dentro del desarrollo de sistemas de información, ya que determina el grado de confiabilidad, estabilidad y satisfacción que experimentarán los usuarios durante la utilización de la plataforma. En el caso de PointCheck, el aseguramiento de la calidad no fue considerado como una actividad aislada al finalizar el proyecto, sino como un proceso continuo desarrollado paralelamente a cada Sprint.

Desde las primeras etapas del desarrollo se estableció como objetivo mantener un sistema funcional después de cada iteración, verificando permanentemente que las nuevas funcionalidades no afectaran el comportamiento de los módulos previamente implementados. Esta estrategia permitió detectar errores de forma temprana, disminuir el costo asociado a su corrección y mantener una evolución estable del proyecto.

El proceso de aseguramiento de la calidad contempló actividades de revisión de código, validación de reglas de negocio, pruebas funcionales sobre la aplicación móvil, verificación de la integración entre frontend y backend y comprobación de la consistencia de la información almacenada en la base de datos.

Adicionalmente, cada proceso de refactorización realizado durante el desarrollo fue acompañado de una validación completa de los módulos afectados, asegurando que la reorganización arquitectónica no modificara el comportamiento esperado del sistema.

La utilización de GitHub también permitió controlar la calidad del código mediante ramas independientes destinadas al desarrollo de nuevas funcionalidades y pruebas experimentales antes de integrar los cambios a la rama principal, reduciendo considerablemente el riesgo de incorporar errores críticos al proyecto.

## 7.2 Estrategia de Verificación

El proceso de verificación se organizó considerando los distintos componentes que conforman la arquitectura del sistema.

En primer lugar, se validó el correcto funcionamiento del backend verificando la respuesta de cada endpoint implementado, comprobando la correcta ejecución de la lógica de negocio y la persistencia adecuada de la información.

Posteriormente se realizaron pruebas sobre la aplicación móvil, verificando la navegación entre pantallas, la representación correcta de los datos recibidos desde el backend y el comportamiento de la interfaz frente a distintos escenarios de uso.

Finalmente se verificó la integración completa entre ambas capas, comprobando que la información enviada desde la aplicación fuera correctamente procesada por el servidor y posteriormente almacenada dentro de la base de datos.

Este enfoque permitió validar el sistema desde una perspectiva integral, reproduciendo situaciones similares a las que enfrentaría un usuario en un entorno real de utilización.

## 7.3 Casos de Prueba Funcionales

Con el propósito de verificar el cumplimiento de los requerimientos definidos para el proyecto, se diseñó un conjunto de **50 casos de prueba funcionales**, orientados a validar cada uno de los módulos implementados durante el desarrollo.

Las pruebas fueron construidas considerando escenarios de uso reales, incluyendo tanto situaciones esperadas de funcionamiento como casos de error destinados a evaluar la robustez del sistema frente a entradas inválidas o condiciones excepcionales.

Entre los principales módulos sometidos a pruebas se encuentran:

- Registro e inicio de sesión de usuarios.
- Gestión de perfiles profesionales.
- Administración de servicios.
- Motor de reservas.
- Registro de atenciones.
- Facturación.
- Dashboard.
- Reportes.
- Gestión de usuarios.
- Auditoría.
- Configuración general.
- Integración con APIs externas.

Cada caso de prueba definió de manera explícita el objetivo de la validación, las condiciones iniciales necesarias para su ejecución, los pasos realizados por el usuario, el resultado esperado y el resultado obtenido durante la ejecución.

Esta metodología permitió mantener un registro detallado del comportamiento del sistema y facilitar futuras actividades de mantenimiento.

## 7.4 Errores Detectados Durante el Desarrollo

El desarrollo iterativo del proyecto permitió identificar diversos problemas técnicos que fueron corregidos progresivamente conforme avanzaban los Sprints.

Uno de los primeros inconvenientes detectados correspondía a la organización del código heredada del prototipo inicial, donde múltiples responsabilidades se encontraban concentradas dentro de un mismo componente. Esta situación dificultaba considerablemente el mantenimiento y aumentaba el riesgo de introducir errores al modificar funcionalidades existentes.

Como respuesta a esta problemática se implementó una refactorización completa basada en una arquitectura modular por funcionalidades, reduciendo significativamente el acoplamiento entre componentes.

Durante el desarrollo del módulo de reservas también se detectaron inconsistencias relacionadas con la validación de disponibilidad horaria, permitiendo inicialmente registrar citas superpuestas. Esta situación fue corregida incorporando validaciones adicionales dentro de la lógica de negocio del backend antes de confirmar una nueva reserva.

Otro aspecto relevante estuvo relacionado con la generación de reportes financieros. En versiones iniciales, los cálculos eran realizados completamente en memoria utilizando filtros sobre colecciones de Kotlin, provocando un consumo innecesario de recursos cuando aumentaba la cantidad de registros almacenados.

La solución consistió en trasladar estas operaciones directamente hacia MySQL mediante consultas JPQL y SQL nativo, mejorando considerablemente el rendimiento general del sistema.

Asimismo, durante la implementación del módulo de autenticación se detectó que las contraseñas eran comparadas directamente como texto plano. Esta situación representaba un riesgo importante para la seguridad de la información y fue solucionada mediante la incorporación de Spring Security y BCrypt para el almacenamiento cifrado de credenciales.

Finalmente, la revisión continua del proyecto permitió simplificar diversos flujos de navegación dentro de la aplicación, eliminando rutas que requerían el envío innecesario de múltiples identificadores y reduciendo la posibilidad de manipulación maliciosa de parámetros por parte de usuarios externos.

## 7.5 Validación de Integración

Uno de los principales objetivos del Proyecto APT consistía en lograr la integración completa entre la aplicación móvil y el backend desarrollado en Spring Boot.

Durante las pruebas realizadas se verificó el correcto funcionamiento de todos los procesos que requieren intercambio de información entre ambas capas.

Entre ellos destacan:

- Registro de usuarios.
- Inicio de sesión.
- Consulta de especialistas.
- Administración de servicios.
- Creación de reservas.
- Registro de atenciones.
- Generación automática de cobros.
- Visualización de dashboards.
- Descarga de reportes.
- Consulta de auditorías.

Cada una de estas operaciones fue ejecutada verificando que la información enviada desde la aplicación fuera correctamente procesada por el backend y posteriormente almacenada dentro de la base de datos sin pérdida de información ni inconsistencias.

Los resultados obtenidos demostraron una integración estable entre todos los componentes del sistema.

## 7.6 Resultados del Proceso de Calidad

Como resultado del proceso de aseguramiento de calidad fue posible validar el correcto funcionamiento de la totalidad de las funcionalidades comprometidas para el Proyecto APT.

Las pruebas ejecutadas permitieron comprobar que el sistema responde adecuadamente frente a distintos escenarios de utilización, manteniendo la integridad de la información, la estabilidad de la plataforma y una experiencia de usuario consistente.

Asimismo, la incorporación de mecanismos de seguridad, validaciones de negocio y mejoras arquitectónicas permitió aumentar considerablemente la confiabilidad del sistema respecto del prototipo inicial.

# CAPÍTULO VIII. RESULTADOS OBTENIDOS

## 8.1 Evolución del Proyecto

Uno de los resultados más importantes obtenidos durante el desarrollo de PointCheck corresponde a la transformación completa del prototipo inicial en una plataforma integral preparada para responder a necesidades reales de organizaciones dedicadas a la prestación de servicios.

Al inicio del proyecto, la aplicación presentaba una arquitectura limitada, funcionalidades parcialmente implementadas y una estructura que dificultaba el mantenimiento del software. A medida que avanzaron los Sprints, el equipo logró reorganizar completamente la solución, incorporar nuevas tecnologías, fortalecer la seguridad del sistema y ampliar significativamente las capacidades funcionales de la plataforma.

Esta evolución quedó reflejada tanto en la arquitectura implementada como en el incremento progresivo de funcionalidades disponibles para cada tipo de usuario.

## 8.2 Resultados Técnicos

Desde una perspectiva técnica, el proyecto alcanzó la totalidad de los objetivos definidos durante la planificación inicial.

Se desarrolló un backend robusto utilizando Spring Boot, organizado mediante una arquitectura basada en funcionalidades que facilita el mantenimiento y futuras ampliaciones del sistema.

La aplicación móvil fue completamente modernizada utilizando Kotlin y Jetpack Compose, incorporando el patrón MVVM para garantizar una adecuada separación entre la interfaz de usuario y la lógica de negocio.

La base de datos evolucionó hacia un modelo relacional más consistente, incorporando nuevas entidades, relaciones optimizadas y el uso de identificadores UUID como mecanismo de identificación seguro.

Adicionalmente, la integración con OpenWeather y Google Maps permitió enriquecer la experiencia de usuario mediante funcionalidades que aportan valor real al funcionamiento cotidiano de la plataforma.

## 8.3 Resultados Funcionales

La versión final de PointCheck incorpora un conjunto completo de funcionalidades distribuidas según el rol del usuario.

Los clientes pueden registrarse, iniciar sesión, buscar especialistas, reservar servicios, consultar el estado de sus citas y revisar su historial de atenciones.

Los especialistas disponen de herramientas para administrar servicios, configurar horarios, gestionar reservas, registrar atenciones, visualizar indicadores de desempeño y generar reportes financieros.

Por su parte, los administradores cuentan con funcionalidades para supervisar usuarios, administrar configuraciones globales, acceder a registros de auditoría y controlar el funcionamiento general de la plataforma.

Estas funcionalidades conforman un flujo completo de gestión que cubre todas las etapas involucradas en la prestación de servicios.

## 8.4 Resultados en Seguridad

Uno de los avances más significativos del proyecto corresponde al fortalecimiento de la seguridad de la información.

La implementación de BCrypt permitió eliminar completamente el almacenamiento de contraseñas en texto plano.

La incorporación de control de acceso basado en roles restringe correctamente las operaciones disponibles para cada tipo de usuario.

La utilización de UUID reduce significativamente la posibilidad de ataques basados en enumeración de recursos.

La implementación de auditoría proporciona trazabilidad completa sobre las acciones críticas realizadas dentro del sistema.

Finalmente, el uso de transacciones garantiza la consistencia de la información incluso frente a errores inesperados durante la ejecución de procesos complejos.

Estas mejoras alinean el desarrollo con las recomendaciones establecidas por la norma ISO/IEC 27001 y preparan la plataforma para futuras ampliaciones relacionadas con seguridad empresarial.



## 8.5 Impacto del Proyecto

Más allá de los resultados técnicos obtenidos, PointCheck representa una solución con potencial de aplicación dentro del mercado de pequeñas y medianas empresas que administran servicios mediante reservas.

La plataforma permite digitalizar procesos que tradicionalmente son ejecutados de manera manual, reduciendo tiempos administrativos, minimizando errores humanos y proporcionando información en tiempo real para apoyar la toma de decisiones.

Asimismo, la arquitectura implementada facilita la incorporación futura de nuevas funcionalidades, permitiendo adaptar el sistema a distintos tipos de negocios sin necesidad de modificar su estructura principal.

Desde el punto de vista académico, el proyecto permitió integrar conocimientos relacionados con ingeniería de software, arquitectura de sistemas, bases de datos, desarrollo móvil, desarrollo backend, seguridad informática, integración de servicios, metodologías ágiles y aseguramiento de la calidad, constituyendo una experiencia completa de desarrollo similar a la enfrentada por equipos profesionales dentro de la industria del software.

## 8.6 Cumplimiento de los Objetivos

Tras la finalización del Proyecto APT se concluye que el objetivo general fue alcanzado satisfactoriamente, desarrollando una plataforma funcional, escalable y segura que integra una aplicación móvil Android con un backend basado en Spring Boot.

Asimismo, todos los objetivos específicos fueron cumplidos mediante la implementación de una arquitectura modular, la incorporación de mecanismos de seguridad, la integración de servicios externos, el desarrollo de funcionalidades diferenciadas por rol, la optimización del rendimiento del sistema y la ejecución de un proceso continuo de verificación de calidad.

Los resultados obtenidos demuestran que PointCheck evolucionó desde un prototipo inicial hacia una solución integral preparada para responder a requerimientos reales del ámbito profesional, cumpliendo los compromisos establecidos para el Proyecto APT y evidenciando el dominio de las competencias técnicas propias de la Ingeniería en Informática.

# CAPÍTULO VIII. RESULTADOS OBTENIDOS

## 8.1 Resultados Generales del Proyecto

La ejecución del Proyecto APT permitió transformar un prototipo inicial con funcionalidades limitadas en una plataforma integral para la administración de reservas, atención de clientes y gestión de profesionales denominada **PointCheck**. El desarrollo realizado durante el período comprendido entre abril y junio de 2026 evidenció una evolución progresiva tanto desde el punto de vista técnico como funcional, incorporando mejoras significativas en la arquitectura del sistema, la organización del código, la seguridad de la información y la experiencia de usuario.

Uno de los principales resultados alcanzados fue la consolidación de una solución completamente integrada bajo un modelo cliente-servidor, donde la aplicación móvil Android interactúa de forma transparente con un backend desarrollado en Spring Boot mediante servicios REST. Esta integración permitió automatizar procesos que originalmente eran ejecutados de forma manual o mediante herramientas independientes, proporcionando una plataforma unificada capaz de gestionar el ciclo completo de atención de un servicio.

El proyecto también permitió demostrar la viabilidad de aplicar metodologías ágiles dentro de un contexto académico, organizando el trabajo mediante Sprints de dos semanas, planificación incremental y control de versiones utilizando GitHub. Esta metodología facilitó la coordinación del equipo, permitió mantener una evolución continua del software y dejó evidencia del proceso de desarrollo mediante el historial de commits generado durante toda la ejecución del proyecto.

## 8.2 Resultados Técnicos

Desde el punto de vista tecnológico, PointCheck alcanzó un nivel de desarrollo considerablemente superior al prototipo original. Una de las mejoras más relevantes fue la migración hacia una arquitectura **Feature-Based**, reorganizando completamente tanto el backend como la aplicación móvil.

Esta decisión permitió reducir el acoplamiento entre módulos, facilitar el mantenimiento del software y simplificar la incorporación de nuevas funcionalidades. La modularización también mejoró el trabajo colaborativo del equipo, ya que permitió desarrollar distintos componentes de manera paralela sin generar conflictos importantes durante la integración del código.

El backend fue consolidado utilizando Spring Boot como plataforma principal para la implementación de la lógica de negocio. Se desarrollaron múltiples módulos independientes encargados de administrar autenticación, usuarios, especialistas, reservas, servicios, atención de clientes, facturación, reportes, auditoría y configuración general del sistema. La utilización de Spring Data JPA permitió simplificar el acceso a los datos y garantizar la consistencia de las operaciones realizadas sobre la base de datos.

En la aplicación móvil se implementó el patrón arquitectónico MVVM utilizando Kotlin y Jetpack Compose. Esta combinación permitió desarrollar interfaces modernas, reutilizables y desacopladas de la lógica de negocio, mejorando considerablemente la mantenibilidad del proyecto y reduciendo la duplicidad de código existente en el prototipo inicial.

Asimismo, la implementación de almacenamiento local mediante Room permitió optimizar la experiencia del usuario almacenando información frecuentemente utilizada, reduciendo tiempos de carga y permitiendo que determinadas funcionalidades permanecieran disponibles incluso ante interrupciones temporales de la conectividad.

## 8.3 Resultados Funcionales

La versión final de PointCheck incorpora un conjunto completo de funcionalidades distribuidas según el perfil del usuario, permitiendo cubrir el flujo completo de atención requerido por organizaciones dedicadas a la prestación de servicios.

Para los clientes se desarrolló un proceso completo de registro, autenticación y gestión de reservas. Los usuarios pueden consultar especialistas disponibles, visualizar información detallada de cada profesional, seleccionar servicios, reservar citas según disponibilidad horaria y consultar posteriormente el estado de sus reservas e historial de atenciones.

Los especialistas cuentan con herramientas orientadas a la administración de su actividad profesional, incluyendo la configuración de perfiles comerciales, administración del catálogo de servicios, definición de horarios laborales, gestión de reservas, registro de atenciones, visualización de indicadores de desempeño y generación de reportes financieros exportables.

Por otra parte, el administrador dispone de un conjunto de herramientas destinadas al control global de la plataforma, permitiendo administrar usuarios, gestionar configuraciones generales, supervisar registros de auditoría, controlar el estado operativo del sistema y acceder a información consolidada relacionada con la actividad financiera de la aplicación.

La integración de estas funcionalidades permite que PointCheck cubra de forma integral todas las etapas involucradas en la prestación de un servicio, desde el primer contacto del cliente hasta el análisis administrativo posterior.

## 8.4 Resultados en Seguridad y Calidad

Uno de los avances más significativos obtenidos durante el desarrollo corresponde al fortalecimiento de la seguridad de la información.

La implementación de Spring Security permitió incorporar mecanismos modernos de autenticación y autorización, eliminando prácticas inseguras presentes en versiones anteriores del sistema. Las contraseñas comenzaron a almacenarse utilizando algoritmos de hash BCrypt, evitando completamente la persistencia de credenciales en texto plano.

La utilización de identificadores UUID mejoró la protección frente a ataques de enumeración de recursos, mientras que la incorporación de registros de auditoría permitió mantener trazabilidad completa sobre las acciones críticas realizadas por los distintos usuarios de la plataforma.

Asimismo, el uso de operaciones transaccionales mediante `@Transactional` permitió garantizar la integridad de la información durante procesos complejos, evitando inconsistencias cuando una operación requiere modificar múltiples entidades de manera simultánea.

Estas mejoras fueron implementadas siguiendo principios inspirados en la norma ISO/IEC 27001, fortaleciendo la confidencialidad, integridad y disponibilidad de la información administrada por el sistema.

## 8.5 Resultados del Proceso de Verificación

Como parte del aseguramiento de calidad se diseñó y ejecutó un conjunto de cincuenta casos de prueba funcionales destinados a validar los principales módulos implementados durante el proyecto.

Las pruebas abarcaron procesos de autenticación, administración de usuarios, reservas, atención de clientes, facturación, generación de reportes, dashboards, integración con APIs externas y mecanismos de seguridad.

Los resultados obtenidos permitieron identificar diversas oportunidades de mejora durante el desarrollo, las cuales fueron corregidas progresivamente antes de la entrega de la versión final del sistema. Entre las principales correcciones realizadas destacan la optimización del motor de reservas, mejoras en la navegación entre pantallas, fortalecimiento de la validación de datos, optimización de consultas SQL y simplificación de distintos procesos internos del backend.

Como resultado de estas actividades, la versión final de PointCheck presentó un comportamiento estable y consistente frente a los distintos escenarios de uso considerados durante las pruebas.

## 8.6 Impacto Profesional del Proyecto

Más allá de los resultados técnicos obtenidos, PointCheck representa una solución con un importante potencial de aplicación dentro del mercado de pequeñas y medianas empresas.

La plataforma permite digitalizar procesos administrativos que tradicionalmente son realizados mediante agendas físicas, hojas de cálculo o aplicaciones de mensajería, contribuyendo a disminuir errores operacionales, mejorar la organización interna y facilitar la toma de decisiones mediante información disponible en tiempo real.

El proyecto también evidencia la aplicación integrada de múltiples competencias propias de la Ingeniería en Informática, incluyendo análisis de requerimientos, diseño de arquitectura de software, desarrollo móvil, implementación de APIs REST, modelamiento de bases de datos, seguridad informática, metodologías ágiles, aseguramiento de calidad y documentación técnica.

En consecuencia, PointCheck constituye una solución técnicamente sólida, preparada para continuar evolucionando y capaz de responder a necesidades reales del ámbito profesional.

# CAPÍTULO IX. CONCLUSIONES Y TRABAJO FUTURO

## 9.1 Conclusiones

El desarrollo de PointCheck permitió cumplir satisfactoriamente los objetivos definidos al inicio del Proyecto APT, logrando transformar un prototipo inicial en una plataforma integral para la gestión de reservas, atención de clientes y administración de profesionales. A lo largo del proyecto fue posible aplicar conocimientos adquiridos durante la formación académica, integrando distintas áreas de la Ingeniería en Informática dentro de una solución tecnológica unificada y funcional.

Uno de los principales aprendizajes obtenidos corresponde a la importancia que posee una arquitectura de software correctamente diseñada desde las primeras etapas del desarrollo. El análisis realizado sobre el prototipo evidenció que una estructura poco organizada dificulta considerablemente la incorporación de nuevas funcionalidades y aumenta el costo de mantenimiento del sistema. La migración hacia una arquitectura basada en funcionalidades permitió superar estas limitaciones, demostrando que una adecuada organización del código constituye un elemento clave para garantizar la evolución sostenible de un proyecto de software.

La utilización de Scrum como metodología de trabajo también representó un aporte significativo para el desarrollo del proyecto. La planificación mediante Sprints permitió distribuir el trabajo de manera equilibrada, mantener una comunicación permanente entre los integrantes del equipo y realizar ajustes oportunos conforme evolucionaban los requerimientos. Esta forma de trabajo facilitó la incorporación progresiva de mejoras sin afectar la estabilidad general del sistema.

Desde el punto de vista técnico, el proyecto permitió consolidar competencias relacionadas con desarrollo backend utilizando Spring Boot, desarrollo móvil mediante Kotlin y Jetpack Compose, diseño de bases de datos relacionales, integración de APIs REST, control de versiones mediante GitHub y aplicación de principios modernos de seguridad informática. La integración de estas tecnologías permitió desarrollar una plataforma robusta, modular y preparada para futuras ampliaciones.

La incorporación de mecanismos de seguridad inspirados en la norma ISO/IEC 27001 fortaleció considerablemente la protección de la información administrada por el sistema. La implementación de BCrypt, UUID, registros de auditoría, control de acceso basado en roles y operaciones transaccionales demuestra la importancia de considerar la seguridad como un componente transversal del desarrollo y no como una funcionalidad adicional incorporada al finalizar el proyecto.

Otro aspecto relevante corresponde al proceso de aseguramiento de calidad. La ejecución de pruebas funcionales durante cada Sprint permitió identificar errores tempranamente y validar el comportamiento del sistema frente a distintos escenarios de uso. Esta estrategia contribuyó significativamente a la estabilidad alcanzada por la versión final de la plataforma.

Finalmente, PointCheck constituye una solución con potencial de utilización dentro del ámbito profesional, permitiendo digitalizar procesos administrativos asociados a organizaciones que trabajan mediante reservas de servicios. Su arquitectura modular, la calidad de las tecnologías implementadas y la documentación generada durante el proyecto proporcionan una base sólida para continuar evolucionando la plataforma mediante futuras etapas de desarrollo.

## 9.2 Trabajo Futuro

Aunque los objetivos definidos para el Proyecto APT fueron alcanzados satisfactoriamente, la arquitectura implementada permite identificar diversas oportunidades de evolución que podrían incorporarse en futuras versiones de PointCheck.

Una de las mejoras más relevantes corresponde a la integración de una pasarela de pagos que permita realizar el cobro de servicios directamente desde la aplicación mediante plataformas como Transbank Webpay, Mercado Pago o Stripe. Esta funcionalidad permitiría automatizar completamente el proceso financiero y reducir la intervención manual durante la gestión de cobros.

Otra línea de desarrollo corresponde a la incorporación de notificaciones Push utilizando Firebase Cloud Messaging. Mediante este mecanismo sería posible recordar automáticamente citas próximas, informar cambios de horario, confirmar reservas y comunicar novedades importantes a los usuarios de la plataforma.

También resulta pertinente desarrollar una versión web administrativa utilizando tecnologías modernas como React o Angular. Esta versión permitiría administrar el sistema desde computadores de escritorio, facilitando tareas relacionadas con reportes, análisis de indicadores y configuración general de la plataforma.

Desde el punto de vista de la seguridad, futuras versiones podrían incorporar autenticación multifactor (MFA), gestión avanzada de sesiones, cifrado de información sensible almacenada en la base de datos y monitoreo continuo de eventos de seguridad, fortaleciendo aún más el cumplimiento de los controles establecidos por la norma ISO/IEC 27001.



En el ámbito analítico, la incorporación de herramientas de inteligencia artificial permitiría generar recomendaciones automáticas para la gestión de agendas, predicción de demanda, optimización de horarios laborales y análisis del comportamiento de los clientes, agregando un importante valor estratégico para las organizaciones que utilicen la plataforma.

Finalmente, la naturaleza modular de PointCheck facilita la incorporación de nuevas funcionalidades específicas para distintos sectores económicos, permitiendo adaptar la plataforma a clínicas médicas, centros odontológicos, talleres mecánicos, gimnasios, centros de estética, asesorías profesionales y cualquier organización cuya operación dependa de la administración eficiente de reservas.

En consecuencia, el trabajo desarrollado durante este Proyecto APT no representa el término del ciclo de vida del software, sino el establecimiento de una base tecnológica sólida sobre la cual pueden construirse futuras versiones con un alcance funcional aún mayor, consolidando a PointCheck como una plataforma escalable y preparada para responder a las necesidades cambiantes del mercado de servicios.

# CAPÍTULO X. COMPETENCIAS DE EMPLEABILIDAD

El desarrollo de PointCheck no solo permitió aplicar conocimientos técnicos propios de la Ingeniería en Informática, sino también fortalecer competencias transversales fundamentales para el desempeño profesional dentro de equipos de desarrollo de software. Durante la ejecución del Proyecto APT fue necesario organizar el trabajo colaborativo, distribuir responsabilidades, gestionar tiempos, resolver problemas técnicos y tomar decisiones de manera conjunta, reproduciendo un entorno similar al que enfrenta un equipo de desarrollo en una organización.

Estas competencias fueron desarrolladas de manera progresiva a lo largo de los distintos Sprints y constituyen un elemento esencial para comprender el proceso completo de ejecución del proyecto.

## 10.1 Trabajo en Equipo

El desarrollo de PointCheck fue realizado por un equipo compuesto por tres integrantes, quienes trabajaron bajo una metodología ágil basada en Scrum. Desde el inicio del proyecto se definieron responsabilidades específicas para cada integrante, procurando distribuir las tareas de acuerdo con las fortalezas técnicas de cada uno y favoreciendo el trabajo paralelo sobre distintos módulos del sistema.

La coordinación del equipo se realizó mediante reuniones periódicas de planificación, revisión y seguimiento. En estas instancias se evaluaban los avances obtenidos durante cada Sprint, se identificaban dificultades técnicas y se definían las prioridades correspondientes al siguiente ciclo de desarrollo. Esta dinámica permitió mantener una comunicación constante entre los integrantes, facilitando la integración del trabajo realizado y reduciendo la aparición de conflictos durante el proceso de desarrollo.

El uso de GitHub como plataforma de control de versiones desempeñó un papel fundamental para el trabajo colaborativo. La utilización de ramas independientes permitió desarrollar nuevas funcionalidades, realizar procesos de refactorización y efectuar pruebas experimentales sin afectar la estabilidad de la rama principal del proyecto. Posteriormente, mediante procesos de revisión e integración, los cambios fueron incorporados de manera controlada, garantizando la consistencia del código desarrollado por el equipo.

La distribución de responsabilidades permitió abordar simultáneamente distintos frentes de trabajo. Mientras algunos integrantes concentraban sus esfuerzos en la evolución del backend y la implementación de la lógica de negocio, otros desarrollaban la aplicación móvil, optimizaban la experiencia de usuario o actualizaban la documentación técnica. Esta organización favoreció un avance continuo del proyecto y permitió cumplir los plazos establecidos durante la planificación inicial.

Además de la coordinación técnica, el trabajo colaborativo favoreció la discusión de distintas alternativas de solución frente a problemas complejos, permitiendo seleccionar aquellas opciones que ofrecían mejores resultados desde el punto de vista de la escalabilidad, mantenibilidad y seguridad del sistema.

Como resultado de este proceso, el equipo logró desarrollar una solución integrada, manteniendo una visión común del proyecto y demostrando la capacidad para trabajar de forma organizada, autónoma y orientada al cumplimiento de objetivos compartidos.

## **10.2 Resolución de Problemas**

Durante el desarrollo de PointCheck surgieron múltiples desafíos técnicos que requirieron procesos de análisis, investigación y toma de decisiones orientados a mantener la calidad del sistema.

Uno de los primeros problemas identificados correspondía a la arquitectura heredada del prototipo inicial. La estructura existente dificultaba considerablemente el mantenimiento del software debido al elevado acoplamiento entre componentes y a la escasa separación entre responsabilidades. Frente a esta situación se decidió reorganizar completamente la solución mediante una arquitectura basada en funcionalidades, proceso que implicó una importante refactorización tanto del backend como de la aplicación móvil.

Otro desafío importante estuvo relacionado con el rendimiento del sistema durante la generación de reportes. Inicialmente, los cálculos financieros eran realizados utilizando operaciones sobre colecciones almacenadas en memoria, situación que provocaba un consumo innecesario de recursos conforme aumentaba el volumen de información. La solución implementada consistió en trasladar dichos cálculos hacia consultas ejecutadas directamente por el motor de base de datos mediante JPQL y SQL nativo, mejorando significativamente el rendimiento de la plataforma.

También fue necesario resolver problemas asociados a la seguridad de la información. La versión inicial del sistema almacenaba contraseñas en texto plano y utilizaba identificadores numéricos secuenciales, prácticas que representaban riesgos importantes desde el punto de vista de la seguridad informática. Como respuesta se incorporaron mecanismos de cifrado mediante BCrypt, identificadores UUID, control de acceso basado en roles y registros de auditoría para fortalecer la protección de la información administrada por el sistema.

Otro aspecto relevante fue la optimización del flujo de navegación entre pantallas. Durante las pruebas funcionales se identificaron procesos que requerían el envío innecesario de múltiples parámetros entre actividades, aumentando la complejidad del código y generando potenciales vulnerabilidades relacionadas con la manipulación de identificadores. La solución consistió en simplificar la navegación utilizando únicamente los identificadores estrictamente necesarios, delegando al backend la responsabilidad de recuperar el resto de la información desde la base de datos.

La resolución de estos problemas permitió fortalecer progresivamente la calidad del proyecto y demuestra la capacidad del equipo para analizar situaciones complejas, evaluar distintas alternativas y aplicar soluciones técnicamente fundamentadas.

## **10.3 Aprendizajes Profesionales**

El desarrollo del Proyecto APT representó una experiencia significativa de aprendizaje que permitió integrar conocimientos adquiridos durante toda la formación académica dentro de un proyecto de características similares a las enfrentadas por equipos profesionales de desarrollo de software.

La implementación de una arquitectura modular, el trabajo colaborativo mediante GitHub, la aplicación de metodologías ágiles, la integración de servicios REST, el desarrollo de aplicaciones móviles modernas y la incorporación de mecanismos avanzados de seguridad permitieron fortalecer competencias técnicas altamente demandadas por la industria informática.

Asimismo, el proyecto evidenció la importancia de mantener una adecuada documentación durante todas las etapas del desarrollo. La elaboración de diagramas, modelos de datos, planificación mediante Sprints, casos de prueba y documentación técnica facilitó considerablemente el seguimiento del proyecto y proporcionó evidencia objetiva del trabajo realizado.

Finalmente, la experiencia adquirida permitió comprender que el desarrollo de software no depende únicamente de la implementación de código, sino también de procesos de planificación, comunicación, aseguramiento de calidad, documentación, control de versiones y mejora continua, aspectos fundamentales para el éxito de proyectos de mayor complejidad.

# REFERENCIAS BIBLIOGRÁFICAS

Las referencias bibliográficas utilizadas para el desarrollo del proyecto se presentan de acuerdo con las normas APA Séptima Edición.

Android Developers. (2025). *Jetpack Compose Documentation*. <https://developer.android.com/jetpack/compose>

Android Developers. (2025). *Architecture Components*. <https://developer.android.com/topic/architecture>

Android Developers. (2025). *Room Persistence Library*. <https://developer.android.com/training/data-storage/room>

Google. (2025). *Google Maps Platform Documentation*. <https://developers.google.com/maps>

OpenWeather. (2025). *OpenWeather API Documentation*. <https://openweathermap.org/api>

Oracle. (2025). *Java Platform Documentation*. <https://docs.oracle.com>

ISO. (2022). *ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. International Organization for Standardization.

Kotlin Foundation. (2025). *Kotlin Language Documentation*. <https://kotlinlang.org/docs/home.html>

MySQL. (2025). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc>

Pivotal Software. (2025). *Spring Boot Documentation*. <https://docs.spring.io/spring-boot>

Spring. (2025). *Spring Security Reference Documentation*. <https://docs.spring.io/spring-security>

Spring. (2025). *Spring Data JPA Reference Documentation*. <https://docs.spring.io/spring-data/jpa>

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. Scrum.org.

Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Pressman, R., & Maxim, B. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

# ANEXOS

Los siguientes anexos complementan la información presentada en el cuerpo principal del informe y constituyen evidencia del proceso de desarrollo del Proyecto APT.

## **Anexo A. [Modelo de Datos](#)**

Se incorpora el diagrama entidad-relación definitivo de la base de datos, incluyendo todas las entidades implementadas, relaciones, claves primarias, claves foráneas y restricciones utilizadas por el sistema.

## **Anexo B. [Arquitectura del Sistema](#)**

Incluye los diagramas correspondientes a la arquitectura general implementada.

## **Anexo C. [Casos de Prueba](#)**

Se adjunta el conjunto completo de **50 casos de prueba funcionales** desarrollados para validar el comportamiento del sistema.

## **Anexo D. [Planificación Scrum](#)**

Se incorpora la planificación completa del proyecto organizada mediante Sprints.

Este anexo permite evidenciar la evolución continua del proyecto desde el prototipo inicial hasta la versión final.

## **Anexo E. [Evidencias del Desarrollo](#)**

Se presentan capturas de pantalla correspondientes a las principales funcionalidades implementadas.

## **Anexo F. [Evidencias del Repositorio](#)**

Se incorporan antecedentes del proceso de desarrollo obtenidos desde GitHub.